Broadcast-Optimal Secure Computation From Black-Box Oblivious Transfer

Michele Ciampi*[1], Divya Ravi†[2], Luisa Siniscalchi[3], and Yu Xia[1]

[1] The University of Edinburgh, UK; michele.ciampi@ed.ac.uk, xy19951128@gmail.com
[2] University of Amsterdam, Netherlands; d.ravi@uva.nl
[3] Technical University of Denmark, Denmark; luisi@dtu.dk

Abstract. When investigating the round-complexity of multi-party computation protocols (MPC) protocols, it is common to assume that in each round parties can communicate over broadcast channels. However, broadcast is an expensive resource, and as such its use should be minimized. For this reason, Cohen, Garay, and Zikas (Eurocrypt 2020) investigated the tradeoffs between the use of broadcast in two-round protocols assuming setup and the achievable security guarantees.
Despite the prolific line of research that followed the results of Cohen, Garay, and Zikas, none of the existing results considered the problem of minimizing the use of broadcast while relying in a *black-box* way on the underlying cryptographic primitives.
In this work, we fully characterize the necessary and sufficient requirements in terms of broadcast usage in the *dishonest majority* setting for round-optimal MPC with black-box use of minimal cryptographic assumptions. Our main result shows that to securely realize any functionality with *unanimous abort* in the common reference string model with black-box use of two-round oblivious transfer it is necessary and sufficient for the parties to adhere to the following pattern: in the first two rounds the parties exchange messages over peer-to-peer channels, and in the last round the messages are sent over a broadcast channel. We also extend our results to the correlated randomness setting where we prove that one round of peer-to-peer interaction and one round of broadcast is optimal to evaluate any functionality with unanimous abort.

1 Introduction

Secure Multi-party Computation (MPC) [CDvdG88, GMW87, Yao86] enables a group of parties, who do not trust each other, to collaboratively compute a function over their private inputs while ensuring that no adversary controlling a subset of participants can gain more information than what is revealed by the output

*Michele Ciampi was partially supported by the Input Output Research Hub (IORH) of the University of Edinburgh and by Sunday Group Inc.

†Divya Ravi was supported by the gravitation project Challenges in Cyber Security (CiCS) with file number 024.006.037 which is financed by the Dutch Research Council (NWO).

of the computation. A significant area of research in the multi-party computation domain focuses on understanding and minimizing the round complexity of MPC protocols [ACGJ18, BGJ+18, BL18, CCG+20, GS18, HHPV18, KO04, MW16]. For the case of security against a malicious adversary (who can arbitrarily deviate) that can corrupt up to all but one party, we know that two rounds are necessary and sufficient to realize any functionality in the common reference string (CRS) model. Moreover, we know how to realize these protocols relying on the minimal assumption of oblivious transfer (OT) [GS18, BL18]. However, these results make *non-black-box* (NBB) use of the underlying cryptographic primitives. This motivated a similar line of research that investigated the round complexity of MPC for the case when only *black-box* [4] (BB) access to the cryptographic primitive is possible. Applebaum *et al.* [ABG+20] show that contrary to what happens in the NBB case, in the CRS model, three rounds are necessary to design a black-box MPC (BBMPC) relying on the minimal assumption that OT is available. On the positive side, in [IKSS22b] the authors show that three rounds are also sufficient to realize a secure protocol while making black-box access to OT. In [GIS18, IKSS22a] the authors also show how to circumvent the impossibility result and design two-round protocols assuming that parties have access to a strong form of setup, which in this paper we will generically refer to as *correlated randomness*.

Unfortunately, all the existing round-optimal black-box protocols with unanimous abort assume that each round of communication corresponds to a round of *broadcast* (BC). A broadcast channel, informally, ensures that when a message is sent, all parties receive it unambiguously. In practical scenarios, a broadcast channel can be simulated using $t+1$ rounds of *peer-to-peer* ($P2P$) communication, where t represents the corruption threshold (the maximum number of parties that an adversary can compromise). In fact, $t+1$ rounds are required for any deterministic protocol that implements broadcast [DS83, FL82]. Alternately, broadcast could be realized using a public bulletin board, such as blockchain. As such broadcast represents quite an expensive resource, which can significantly affect the latency of a protocol.

Driven by this motivation, a recent line of works [CGZ20, DMR+21, DRSY23], [CDR+23] studies if it is plausible to minimize the use of broadcast while maintaining an *optimal* round complexity, at the cost of possibly settling for a weaker security guarantee. In particular, Cohen *et al.* [CGZ20] investigate this question in the dishonest majority setting by looking at the standard notion of security with *unanimous* and *selective* abort that we abbreviate with UA and SA, respectively[5]. The notion of unanimous aborts guarantees that either all the honest parties get the output or none of them do. In the case of selective abort, the adversary can decide which honest parties get the output and which honest parties instead abort.

[4] Similar to [ABG+20], the notion of a black-box construction used in this paper (also referred to as a black-box reduction) corresponds to the notion of a *fully black-box* reduction in the taxonomy of [RTV04].

[5] In [CGZ20] the authors also consider the notion of security with identifiable abort.

Cohen *et al.* [CGZ20] show that, when the majority of the parties can be corrupt, to obtain a two-round MPC protocol with unanimous abort in the CRS model, at least one round of communication must be over broadcast. Given that in the black-box case, three rounds are necessary (and sufficient) to obtain a secure protocol with UA from oblivious transfer, we investigate whether the result of [CGZ20] extends to the three-round setting. In particular, we ask the following question.

Question 1. *How many rounds of broadcast are necessary in the CRS model to securely evaluate any functionality (with UA or SA) in a dishonest majority setting, while making only black-box use of two-round oblivious transfer?*

On the positive side, in [CGZ20] the authors show a round-preserving compiler that turns any two-round MPC protocol secure with unanimous abort, into a new MPC protocol that relies on broadcast only in the last round. The authors also argue that the same compiler yields security with selective abort if only $P2P$ channels are available. A natural next step would be to minimize the use of broadcast for round-optimal BBMPC using the same compiler. Unfortunately, even if we start from an MPC protocol that makes black-box use of primitives, the protocol resulting from applying the Cohen *et al.* [CGZ20] compiler is not black-box (more details on this will follow). Hence, in this chapter, we study the following natural question.

Question 2. *Can we design a round-preserving compiler that can turn any round-optimal BBMPC protocol that uses broadcast in every round into one that makes minimal use of broadcast?*

Our Contributions. We answer the first question by showing that in the CRS model, every round-optimal (three-round) protocol that makes black-box use of OT and that satisfies security with UA must use broadcast at least in the last round.

Theorem 1 (informal). *To obtain a round-optimal (three-round)* UA *secure MPC protocol in the CRS model against a dishonest majority that relies on black-box use of two-round OT, parties must send the last protocol messages over a broadcast channel.*

We answer the second question, by designing a novel *round-preserving information theoretic* compiler, that takes any two-round (three-round) BBMPC protocol secure with UA, and turns it into a new protocol that achieves UA which uses broadcast only in the last round. We also argue that the same compiler can be used to obtain SA when no broadcast is available.

In a nutshell, considering our impossibility result, the impossibility of [CGZ20], and the recent round-optimal BBMPC protocols proposed in [IKSS22b, GIS18, IKSS21, IKSS22a] we provide protocols that use the provably minimal number of rounds while making black-box use of minimal cryptographic assumptions. In particular, we prove the following theorems.

Theorem 2 (informal). *One round of broadcast is sufficient to obtain a* UA*-secure three-round (two-round) MPC protocol in the CRS (correlated randomness) model relying on black-box use of OT (PRGs) in the CRS model, against an adversary that can corrupt the majority of the parties.*

Theorem 3 (informal). *Peer-to-peer communication is sufficient to obtain a* SA*-secure three-round MPC protocol in the CRS model relying on black-box use of OT in the CRS model, against an adversary that can corrupt the majority of the parties.*

For a comprehensive summary of our results and comparison with existing works, we refer to Table 1.

Setup	Broadcast pattern	Primitives	Security	Possible?	Shown in
Corr. Rand.	*BC-BC*	BB	UA	✓	[GIS18]
Corr. Rand.	*BC-P2P*	NBB	UA	✗	[CGZ20]
Corr. Rand.	*P2P-BC*	BB	UA	✓	**This work**[6]
Corr. Rand.	*P2P-P2P*	BB	SA	✓	[IKSS21, IKSS22a]
CRS	*BC-BC*	BB	SA	✗	[ABG+20]
CRS	*P2P-BC*	NBB	UA	✓	[CGZ20]
CRS	*P2P-P2P*	NBB	SA	✓	[CGZ20]
CRS	*BC-BC-BC*	BB	UA	✓	[IKSS22b, PS21]
CRS	*BC-BC-P2P*	BB	UA	✗	**This work**
CRS	*P2P-P2P-BC*	BB	UA	✓	**This work**
CRS	*P2P-P2P-P2P*	BB	SA	✓	**This work**

Table 1: We denote the acronym *P2P* (resp. *BC*) to indicate the peer-to-peer (resp. broadcast) channel (e.g., *P2P-BC* means that the first round of the protocol generated by a party is sent over peer-to-peer channels, whereas the second is sent over broadcast). UA stands for unanimous abort, and SA stands for selective abort. To strengthen our results, in the impossibility results, we assume that the *BC* rounds allow peer-to-peer communication as well; in feasibility results, we assume that the *BC* rounds involve only broadcast messages (and no peer-to-peer messages). By combining our compiler with existing round-optimal protocols, we obtain that all our positive results in the CRS model only make black-box use of OT. The positive results in the correlated randomness (Corr. Rand.) model instead hold assuming black-box access to PRGs.

We end this section by recalling that all our results are for the case of unanimous and selective abort. It is an interesting open problem to understand the minimal use of broadcast and black-box cryptographic primitives to achieve security with *identifiable abort* (IA) [AL07]. This notion requires that in the event of abort, all the honest parties agree on the same identity of a corrupted party. This notion is quite desirable, and it seems to require novel and non-trivial techniques. Indeed, even assuming that broadcast is available in every round, the question of designing a black-box protocol with IA that matches the UA lower bound is still open.

[6] In [CGZ20] the authors provide the same result, but only for the secure evaluation of functions in NC1.

1.1 Technical Overview

Impossibility result. We start by showing that a three-round malicious MPC protocol achieving UA in the CRS model that makes only black-box use of two-round OT, must necessarily use broadcast in the last round (even if broadcast is allowed in the first two rounds). In other words, we show that such a protocol cannot exist in the *BC-BC-P2P* communication pattern. At a high level, we assume, toward contradiction, that there exists a three-round protocol in the pattern *BC-BC-P2P* say Π, achieving UA with only black-box use of two-round OT protocol in the CRS model. We gradually modify this protocol to obtain a new two-round protocol (always in the CRS model), which makes black-box use of OT. But as we have recalled in the previous sections [ABG+20, IKSS22b] show that such a protocol cannot exists, hence we have reached our contradiction.

We now provide more detail about our impossibility proof. Let Π be a three-round protocol that makes black-box use of two-round oblivious transfer in the CRS model, and that achieves UA relying on the communication pattern *BC-BC-P2P*. In particular Π is a five-party protocol run by the parties $(P_1, P_2, P_3, P_4, P_5)$ that aim to evaluate the function f. The function f computes the multiplication of the inputs of the first two parties (P_1, P_2), and returns the output to all the parties. We now show how to gradually modify Π to reach a contradiction. We proceeded with two main steps:

Step 1. First, we show that Π can be transformed into a four-party two-round solitary MPC protocol Π'[7] where the party can compute the output without the need of receiving (and sending) the last message.
In more detail, we design a four-party protocol Π' which is run by the parties (P'_1, P'_2, P'_3, P'_4) that aim to compute the function f', where f' outputs the multiplication of the inputs of the first two parties (P'_1, P'_2) only to P'_4. To construct Π' we modify Π by first removing the messages that the first three parties (P_1, P_2 and P_3) send to P_4, P_5 in the last *P2P* round. We claim that in this new protocol, both P_4 and P_5 can still compute the output. The reason, is that P_4, noticing that he did not receive any message in the last round from the first three parties, has only two options, either abort or somehow compute the output. The same holds for P_5. However, because Π is secure with unanimous abort, if P_4 aborts, P_5 must abort as well. However, note that P_4 has no clue about whether P_5 received the last message from (P_1, P_2, P_3) or not, and P_5 does not have additional round to communicate this information to P_4. Hence, we can argue that P_4 must be able to compute the output, because P_5 may have indeed received the last message from all the parties, hence the behavior of P_4 must be consistent with whatever behavior P_5 has. We can also remove the last round messages that go from (P_4, P_5) to the parties (P_1, P_2, P_3). Indeed, P_4 can still trivially compute the output and the only effect that we have by removing these messages is that P_1, P_2 and P_3 may not be able to compute the output anymore.

[7] Protocols where only a single party obtains the output are referred to as *solitary* MPC (introduced in [HIK+19]).

We now denote P_4' as the party that internally emulates the behavior of the modified P_4 and P_5 we have just described (i.e., they do not communicate with P_1, P_2 and P_3 in the last round). For each $i \in [3]$ we let P_i' be the modified P_i that does not send the last round message to both P_4 and P_5. Following the above logic we obtained the protocol $\Pi'(P_1', P_2', P_3', P_4')$, that consists of just two-rounds of broadcast, as the last $P2P$ is no longer used, where only the party P_4' receives the output. Therefore, we can proceed to the next step.

Step 2. We show that Π' can be transformed to a three-party two-round protocol $\overline{\Pi}$ among $(\overline{P}_1, \overline{P}_2, \overline{P}_3)$ computing g with *augmented* semi-honest security[8], where g computes the multiplication of the first two inputs and outputs to all. At a high-level, in this transformation, each among $(\overline{P}_1, \overline{P}_2, \overline{P}_3)$ plays the role of their counterpart (P_1', P_2', P_3') in Π'. Additionally $\overline{P}_1$ plays the role of P_4' 'publicly' by sampling randomness on behalf of P_4' which is sent along with P_4''s messages over broadcast channel. Informally, the fact that P_4' does not have an input and the augmented semi-honest model (where corrupt parties also follow protocol specifications honestly) allow us to argue security of $\overline{\Pi}$.

We have now obtained a three-party two-round protocol with augmented semi-honest security, computing g using only black-box two-round OT. Here we emphasize that g is the external-AND function, which computes the multiplication of the inputs of first two parties, and gives the output to all three parties.

However, in [ABG+20, IKSS22b][9] the authors show that a three-party two-round protocol, which securely computes external-AND function, cannot exist. Hence, we have reached a contradiction.

Feasibility results. We start by considering the problem of designing a two-round protocol that makes black-box use of cryptographic primitives and achieves UA in the communication pattern $P2P$-BC (which we know to be optimal due to [CGZ20]), and then we show how the approach extends to the three-round case $P2P$-$P2P$-BC (which we have just proven to be optimal).

Let Π_{bc} be a protocol that works only over broadcast channels, we want to devise a compiler that turn it into a new protocol that uses broadcast only in the last round. As a starting point, note that if the first round of Π_{bc} is simply replaced with peer-to-peer channels, then the adversary can send different messages (over peer-to-peer channels) to a pair of honest parties in that round

[8] The notion of augmented semi-honest is the same as the semi-honest notion, with the only difference that the adversary can change its input and abort. We recall that every protocol that is malicious secure is also secure with augmented semi-honest security [HL10, p. 28].

[9] In [ABG+20] the authors show that no two-round semi-honest secure protocol can be constructed from black-box two-round semi-honest OT. In [IKSS22b] the authors observe that the same results hold also for the malicious case when black-box access to malicious secure OT is assumed.

and obtain the honest parties' responses in the second round, computed with respect to different first-round messages. This potentially violates security of Π_{bc} as such a scenario would never happen in the original protocol with broadcast in every round (as the honest parties would have a consistent view of the messages sent in the first round).

To ensure that honest parties' responses are obtained only if they have a consistent view of the corrupt parties' messages, the two-round (NBB) construction of Cohen *et al.* [CGZ20] adopts the following trick: In addition to sending the first-round messages over a peer-to-peer channel (as described above), the parties send a garbled circuit which computes their next-round message (by taking as input the first-round messages, and using the hard-coded values of input and randomness of this party) and additively share labels of this garbled circuit. In the subsequent round, parties send the relevant shares based on the first-round messages they received. The main idea is that the labels corresponding to honest parties' garbled circuits can be reconstructed to obtain their second-round messages *only if* the adversary sends the same first-round messages to every honest party.

Note that the above compiler, even if Π_{bc} makes black-box use of cryptographic primitives, would return a NBB scheme, as garbling the next-message function of Π_{bc} would result in a non-black-box use of whatever cryptographic primitive Π_{bc} relies on. To solve this problem, we significantly simplify the above approach as follows. The garbled circuit of a party simply hardwires a randomly chosen bit-string that we will treat as a one-time pad key. Whenever the garbled circuit is queried (it does not matter with what input as long as it is valid) it will return the secret to the caller.

In the first round, the parties will send the first-round messages of the input protocol Π_{bc}, along with the garbled circuit we have just described, and secret share the labels of the garbled circuit exactly as in the compiler of [CGZ20]. In the second round, each party computes the second-round messages of the input protocol according to all the messages received in the first $P2P$ round and broadcasts a one-time pad encryption of the obtained message using the one-time pad key.

A corrupted party can reconstruct the second-round messages of the honest parties only if he has sent the same first-round messages to all the honest parties. Indeed, if this is not the case, then the corrupted party would not be able to retrieve the key hardwired inside the garbled circuits generated by the honest party, hence, he would not be able to decrypt the second-round messages sent by these parties.

We are almost done, but there is still a slight caveat. A corrupt party could send different garbled circuits to different honest parties. This will make the view of honest parties inconsistent with respect to Round 2 of the corrupt party. To address this, we follow the approach of [CDR+23] and use 'broadcast with abort' [GL05] to realize a 'weak' broadcast of garbled circuits over two peer-to-peer rounds – In the first round, as before, each party sends its garbled circuit to others. In the second round, parties additionally echo the garbled circuits

they received in Round 1. A party 'accepts' a garbled circuit only if it has been echoed by all other parties, or else she aborts. This ensures that if a pair of honest parties do not abort, they must have received the same garbled circuit and therefore would have a consistent view of Round 2 of corrupt parties as well.

We note that the overhead introduced by our compiler is quite minimal due to the function being garbled being just a constant function. Moreover, our compiler does not introduce any additional complexity assumption as garbled circuits for constant functions can be realized information-theoretically.

We finally observe that for the pattern $P2P$-$P2P$, instead, we can use exactly the same compiler to obtain security with selective abort. In this case, the adversary could send different second-round messages to different parties, hence making only a subset of honest parties to abort. In this case, however, the compiler works only if Π_{bc} guarantees that the input of the parties is committed in the first round. Indeed, if this is not the case, then the adversary could force different outputs to different parties. However, we note that this property is enjoyed by all known round-optimal constructions.

The three(multi)-round case. This time, we have a three-round protocol Π_{bc} that enjoys UA in the pattern BC-BC-BC, and we want to turn it into a protocol that is secure in the pattern $P2P$-$P2P$-BC, while preserving its security. We can adopt the same technique described above by iterating the garbled-circuit approach also in the third round. We refer to the technical section of the chapter for more detail. We finally note that our compiler does not need any additional setup, on top of whatever it is required by the input protocol. As such, our technique can be easily applied to any black-box protocol, even to the recent black-box round-optimal protocol designed in the plain model [IKSS23, COSW23].

2 Preliminaries

2.1 Notation

We denote "$\leftarrow$" as the assigning operator (e.g., to assign to a the value of b we write $a \leftarrow b$), and "$\xleftarrow{\$}$" as the sample operator (e.g., $x \xleftarrow{\$} Q$ denotes the sampling of x from a distribution Q). We use "$=$" to check equality of two different elements. We use $\mathsf{poly}(\cdot)$ to indicate a generic polynomial function, $\mathtt{negl}(\cdot)$ to denote a negligible function. We use $\stackrel{c}{\equiv}$ (resp. $\stackrel{c}{\not\equiv}$) to denote computational indistinguishability (resp. distinguishability) of two ensembles. We use $\stackrel{p}{\equiv}$ to denote that two ensembles are identical. Let λ be the security parameter. let $\mathbb{N}$ be the set of the natural number.

2.2 Garbling scheme

We use the following definition of the garbling schemes.

Definition 1 (Garbling scheme [BHR12, CGZ20, DMR+21]). *A garbling scheme is a pair of algorithms* ($\mathtt{garble}$, $\mathtt{eval}$)*, such that:*

- $(\mathtt{GC}, \mathbf{K}) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C})$*: on input the unary representation of the security parameter* 1^λ *and a boolean circuit* $\mathtt{C} : \{0,1\}^{\mathsf{L}} \to \{0,1\}^m$*, the garbling algorithm outputs a garbled circuit* $\mathtt{GC}$ *and* L *pairs of garbled labels* $\mathbf{K} = \{\mathbf{K}_\alpha^b\}_{b\in\{0,1\},\alpha\in[\mathsf{L}]}$*. For simplicity, we assume that for every* $\alpha \in [\mathsf{L}]$ *and* $b \in \{0,1\}$*, it holds that* $\mathbf{K}_\alpha^b \in \{0,1\}^\lambda$*.*
- $y \leftarrow \mathtt{eval}(\mathtt{GC}, \{\mathbf{K}_\alpha\}_{\alpha\in[\mathsf{L}]})$*: on input a garbled circuit* $\mathtt{GC}$ *and* L *garbled labels* $\{\mathbf{K}_\alpha\}_{\alpha\in[\mathsf{L}]}$*, the evaluation algorithm output a value* $y \in \{0,1\}^m$*.*

The scheme has the following three properties[10]*:*

- Correctness*: For any boolean circuit* $\mathtt{C} : \{0,1\}^{\mathsf{L}} \to \{0,1\}^m$*, and* $x = (x_1, \ldots, x_{\mathsf{L}}) \in \{0,1\}^{\mathsf{L}}$*, it holds that:*

$$\Pr[\mathtt{eval}(\mathtt{GC}, \mathbf{K}[x]) \neq \mathtt{C}(x)] \leq \mathtt{negl}(\lambda)$$

  where $(\mathtt{GC}, \mathbf{K}) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C})$*, with* $\mathbf{K} = (\mathbf{K}_1^0, \mathbf{K}_1^1, \ldots, \mathbf{K}_{\mathsf{L}}^0, \mathbf{K}_{\mathsf{L}}^1)$ *and* $\mathbf{K}[x] = (\mathbf{K}_1^{x_1}, \ldots, \mathbf{K}_{\mathsf{L}}^{x_{\mathsf{L}}})$*.*
- (Adaptive) Privacy*: There exists a probabilistic polynomial-time (PPT) simulator* $\mathtt{simGC}$*, s.t. for every PPT adversary* $\mathcal{A}$*:*

$$\left| \Pr\left[\mathtt{Expt}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, 0) = 1\right] - \Pr\left[\mathtt{Expt}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, 1) = 1\right] \right| \leq \mathtt{negl}(\lambda)$$

  where the experiment $\mathtt{Expt}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, b)$ *is defined as follows:*
  - *The adversary* $\mathcal{A}$ *specify the boolean circuit* $\mathtt{C} : \{0,1\}^{\mathsf{L}} \to \{0,1\}^m$*.*
  - *If* $b = 0$*, the challenger* $\mathcal{C}$ *computes* $(\mathtt{GC}, \mathbf{K}) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C})$*, and returns* $\mathtt{GC}$*.*
  - *If* $b = 1$*, the challenger* $\mathcal{C}$ *computes* $\mathtt{GC} \leftarrow \mathtt{simGC}(1^\lambda, \phi(\mathtt{C}), \text{“}ckt\text{”})$*, where* $\phi(\mathtt{C})$ *denotes the topology of* $\mathtt{C}$[11]*. The challenger returns* $\mathtt{GC}$*.*
  - *The adversary* $\mathcal{A}$ *specify input* $x = (x_1, \ldots, x_{\mathsf{L}}) \in \{0,1\}^{\mathsf{L}}$*.*
  - *If* $b = 0$*, the challenger* $\mathcal{C}$ *sets* $\mathbf{K}_\alpha = \mathbf{K}_\alpha^{x_\alpha}$*, for* $\alpha \in \mathsf{L}$ *and returns* $\{\mathbf{K}_\alpha\}_{\alpha\in\mathsf{L}}$*.*
  - *If* $b = 1$*, computes* $\mathbf{K}_1, \ldots, \mathbf{K}_{\mathsf{L}} \leftarrow \mathtt{simGC}(1^\lambda, \mathtt{C}(x), \text{“}input\text{”})$*, and returns* $\{\mathbf{K}_\alpha\}_{\alpha\in\mathsf{L}}$*.*
  - $\mathcal{A}$ *outputs a bit* b'*, and it is the output of the experiment.*

- Partial Evaluation Resiliency [DMR+21]: *We say that* $\mathtt{GC}$ *satisfies* partial evaluation resiliency *if there exists a simulator* $\mathtt{simGC}$ *such that for every PPT adversary* $\mathcal{A}$*, it holds that*

$$\left| \Pr\left[\mathtt{Expt}^{\mathsf{PR}}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, 0) = 1\right] - \Pr\left[\mathtt{Expt}^{\mathsf{PR}}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, 1) = 1\right] \right| \leq \mathtt{negl}(\lambda)$$

[10] We also assume that the input labels are authenticated. This can be achieved by using a signature scheme, where each label is signed, and the verification key is sent along with the garbled circuit. Then, during the evaluation phase, the evaluator verifies the signatures on the labels before proceeding with the evaluation of the garbled circuit. If any of the signature verification fails, the output of the evaluator is set to $\perp$.

[11] We assume that the topology of a circuit does not reveal hard coded values (as hard-coded values are essentially fixed input labels for some wires.)

where the experiment $\mathtt{Expt}^{\mathsf{PR}}_{\Pi,\mathcal{A},\mathtt{simGC}}(1^\lambda, b)$ *is defined as follows:*

- *The adversary* $\mathcal{A}$ *specify the boolean* $\mathtt{C} : \{0,1\}^{\mathsf{L}} \to \{0,1\}^m$, $i \in [\mathsf{L}]$, *and* $x = (x_1, \ldots, x_{\mathsf{L}}) \in \{0,1\}^{\mathsf{L}}$
- *If* $b = 0$, *the challenger* $\mathcal{C}$ *computes* $(\mathtt{GC}, \mathbf{K}) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C})$, *and returns* $(\mathtt{GC}, \{\mathbf{K}_\alpha^{x_\alpha}\}_{\alpha\in[\mathsf{L}],\alpha\neq i})$
- *If* $b = 1$, *the challenger* $\mathcal{C}$ *computes* $(\mathtt{GC}, \{\mathbf{K}_\alpha\}_{\alpha\in[\mathsf{L}]}) \leftarrow \mathtt{simGC}(1^\lambda, \mathtt{C})$, *and returns* $(\mathtt{GC}, \{\mathbf{K}_\alpha\}_{\alpha\in[\mathsf{L}],\alpha\neq i})$
- $\mathcal{A}$ *outputs a bit* b', *and it is the output of the experiment.*

Instantiation. As shown by Bellare *et al.* [BHR12], adaptive garbled circuits can be obtained using Yao's garbled circuits and one-time pads. We refer to [DMR+21] for details on how any garbling scheme can be augmented using one-time pads to satisfy partial evaluation resiliency.

2.3 Additive secret sharing scheme

In our work, we will use *n-out-of-n secret sharing scheme*, which is also referred to as *additive secret sharing scheme*. We give the definition here:

Definition 2 (n-out-of-n secret sharing scheme). *A n-out-of-n secret sharing scheme is a pair of algorithms (*`share`, `reconstruct`*), such that:*

- `share` *is a randomized algorithm that on input a secret* m, *outputs a set of* n *shares* $(s_1, \ldots, s_n)$.
- `reconstruct` *is a deterministic algorithm that on input* n *shares* $(s_1, \ldots, s_n)$, *outputs the secret* m.

The scheme needs to satisfy the correctness *property if : For any* m *we have:*

$$\Pr_{\mathtt{share}(m)\to(s_1,\ldots,s_n)}[\mathtt{reconstruct}(s_1, \ldots, s_n) = m] = 1$$

In addition, the scheme satisfies perfect security *property if: for all messages* m, m', $\forall S \subset \{s_1, \ldots, s_n\}$, *the following distributions are indistinguishable:*

$$\{(s_i \mid i \in S) \colon (s_1, \ldots, s_n) \leftarrow \mathtt{share}(m)\}$$
$$\{(s'_i \mid i \in S) \colon (s'_1, \ldots, s'_n) \leftarrow \mathtt{share}(m')\}$$

2.4 Secure Multiparty Computation (MPC) Definitions

In our paper, we follow [CGZ20] to give our MPC definition.

Notation. When we need to specify a n-party function, we define it as $f(x_1, \ldots, x_n) = (y_1, \ldots, y_n)$, where x_i is the party P_i inputs to the function, and y_i is the output received by P_i. If $x_i = \perp$ (resp., $y_i = \perp$), it means that P_i has no input (resp., P_i receives no output).

An n-party protocol $\Pi = (P_1, \ldots, P_n)$ is an n-tuple of probabilistic polynomial-time (PPT) interactive Turing machines (ITMs), where each party P_i is initialized with input $x_i \in \{0,1\}^*$ and random coins $r_i \in \{0,1\}^*$.

The protocol is executed in rounds (i.e., the protocol is synchronous). Each round consists of the send phase and the receive phase, where parties can respectively send the messages from this round to other parties and receive messages from other parties. In every round parties can communicate either over a broadcast channel or a fully connected peer-to-peer network. We assume that these channels are private and we assume the channels to be ideally authenticated.

In this paper, we mainly focus on two-round and three-round secure computation protocols, and we use T to denote the number of rounds. Rather than viewing a protocol Π as an n-tuple of ITMs, it is convenient to view each ITM as a sequence of multiple algorithms: $\texttt{frst-msg}_i$, to compute P_i's first-round messages to its peers; $\texttt{nxt-msg}_i^k$, to compute P_i's k-th round messages for $(2 \leq k \leq T)$; and $\texttt{output}_i$, to compute P_i's output. Thus, a protocol Π can be defined as $\{(\texttt{frst-msg}_i, \texttt{nxt-msg}_i^k, \texttt{output}_i)\}_{i\in[n],k\in[T]\setminus\{1\}}$.

The syntax of the algorithms is as follows:

- $\texttt{frst-msg}_i(x_i; r_i) \rightarrow (\{\mathsf{msg}_{i\rightarrow j}^1\}_{j\in[n]}, \mathsf{msg}_i^1)$ produces the first-round messages of party P_i to all parties. $\mathsf{msg}_{i\rightarrow j}^1$ are the messages to be sent in the peer-to-peer channels, and msg_i^1 are the messages to be sent in the broadcast channels. If the protocol only have access to broadcast channel, then the first-round messages only includes msg_i^1, and if the protocol only have access to peer-to-peer channels, the first-round messages only includes $\{\mathsf{msg}_{i\rightarrow j}^1\}_{j\in[n]}$. Note that a party's message to itself can be considered to be its state.
- $\texttt{nxt-msg}_i^k(x_i, \{\mathsf{msg}_j^l, \mathsf{msg}_{j\rightarrow i}^l\}_{j\in[n],l\in[k-1]}; r_i) \rightarrow (\{\mathsf{msg}_{i\rightarrow j}^k\}_{j\in[n]}, \mathsf{msg}_i^k)$ produces the k-th round messages of party P_i to all parties.
- $\texttt{output}_i(x_i, \{\mathsf{msg}_j^l, \mathsf{msg}_{j\rightarrow i}^l\}_{j\in[n],l\in[T]}; r_i) \rightarrow y_i$ produces the output returned to party P_i.

We note that, unless needed, to not overburden the notation, we do not pass the random coin r as an explicit input of the cryptographic algorithms.

In our paper, we consider the MPC protocol with two different types of setup: the correlated randomness setup and the CRS setup.

- In the correlated randomness model a trusted party samples $(r_1, \ldots, r_n) \xleftarrow{\$} \mathcal{D}_{\mathsf{corr}}$ from some predefined efficiently sampleable distribution $\mathcal{D}_{\mathsf{corr}}$, and each party P_i receives r_i at the onset of the protocol. For simplicity, and without loss of generality, we assume that the random coins of each party are a part of the correlated randomness.
- If the MPC is in the CRS model, then in the setup phase, the CRS is generated by $\mathsf{crs} \xleftarrow{\$} \texttt{setup}(1^\lambda)$, and every algorithm of the MPC protocol takes crs as input additionally.

We denote the acronym BC to indicate a round where broadcast is available and the acronym $P2P$ to indicate a round where only peer-to-peer channels

are available. To strengthen our results, our lower bounds assume that the BC rounds allow peer-to-peer communication as well; our upper bounds assume that the BC rounds involve only broadcast messages (and no peer-to-peer messages).

We define the view of the party as follows:

Definition 3 (View). *The view* view_i *of party* P_i *is defined as the input* x_i*, the randomness* r_i*, and the all the messages* P_i *sends (denoted as* $\mathsf{msg}_{i,\mathsf{out}}$*, where* $\mathsf{msg}_{i,\mathsf{out}} \leftarrow (\{\mathsf{msg}^k_{i\to j}\}_{j\in[n]\setminus\{i\},k\in[T]}, \{\mathsf{msg}^k_i\}_{k\in[T]})$*) and receives (denoted as* $\mathsf{msg}_{i,\mathsf{in}}$*, where* $\mathsf{msg}_{i,\mathsf{in}} \leftarrow (\{\mathsf{msg}^k_{j\to i}\}_{j\in[n]\setminus\{i\},k\in[T]}, \{\mathsf{msg}^k_j\}_{j\in[n]\setminus\{i\},k\in[T]})$*) during the execution of the MPC protocol.*

Malicious security model We follow the real-world/ideal-world simulation paradigm and we adopt the security model of Cohen, Garay and Zikas [CGZ20].

Real-world. We let $\mathcal{A}$ denote a special PPT ITM that represents the adversary and that is initialized with input that contains the identities of the corrupt parties, their respective private inputs, and an auxiliary input.

During the execution of the protocol Π, the corrupt parties receive arbitrary instructions from the adversary $\mathcal{A}$, while the honest parties faithfully follow the instructions of the protocol. We consider the adversary $\mathcal{A}$ to be rushing, i.e., during every round the adversary can see the messages the honest parties sent before producing messages from corrupt parties.

At the end of the protocol execution, the honest parties produce output, and the adversary outputs an arbitrary function of the corrupt parties' view.

Definition 4 (Real-world execution). *Let* $\Pi = (P_1, \ldots, P_n)$ *be an* n*-party protocol and let* $\mathcal{I} \subseteq [n]$*, of size at most* t*, denote the set of indices of the parties corrupted by* $\mathcal{A}$*. The joint execution of* Π *under* $(\mathcal{A}, \mathcal{I})$ *in the real world, on input vector* $\boldsymbol{x} = (x_1, \ldots, x_n)$*, auxiliary input* aux *and the unary representation of the security parameter* λ*, denoted* $\mathtt{REAL}_{\Pi,\mathcal{I},\mathcal{A}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$*, is defined as the output vector of* $P_1, \ldots, P_n$ *and* $\mathcal{A}(\mathsf{aux})$ *resulting from the protocol interaction.*

Ideal-world. We describe ideal world executions with selective abort ($\mathsf{sa\text{-}abort}$), unanimous abort ($\mathsf{un\text{-}abort}$).

Definition 5 (Ideal Computation). *Consider* $\mathsf{type} \in \{\mathsf{sa\text{-}abort}, \mathsf{un\text{-}abort}\}$*. Let* $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ *be an* n*-party function and let* $\mathcal{I} \subseteq [n]$*, of size at most* t*, be the set of indices of the corrupt parties. Then, the joint ideal execution of* f *under* $(\mathcal{S}, \mathcal{I})$ *on input vector* $\boldsymbol{x} = (x_1, \ldots, x_n)$*, auxiliary input* aux *to* $\mathcal{S}$ *and the unary representation of the security parameter* λ*, denoted* $\mathtt{IDEAL}^{\mathsf{type}}_{f,\mathcal{I},\mathcal{S}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$*, is defined as the output vector of* $P_1, \ldots, P_n$ *and* $\mathcal{S}$ *resulting from the following ideal process.*

1. Parties send inputs to trusted party*: An honest party* P_i *sends its input* x_i *to the trusted party. The simulator* $\mathcal{S}$ *may send to the trusted party arbitrary inputs for the corrupt parties. Let* x'_i *be the value actually sent as the input of party* P_i*.*

2. Trusted party speaks to simulator*: The trusted party computes* $(y_1, \ldots, y_n) = f(x'_1, \ldots, x'_n)$*. If there are no corrupt parties proceed to step 4. Otherwise, the trusted party sends* $\{y_i\}_{i\in\mathcal{I}}$ *to* $\mathcal{S}$*.*
3. Simulator $\mathcal{S}$ responds to the trusted party*:*
   (a) If $\mathsf{type} = \mathsf{sa\text{-}abort}$: *The simulator* $\mathcal{S}$ *can select a set of parties that will not get the output as* $\mathcal{H} \subseteq [n] \setminus \mathcal{I}$*. (Note that* $\mathcal{H}$ *can be empty, allowing all parties to obtain the output.) It sends* $(\mathtt{abort}, \mathcal{H})$ *to the trusted party.*
   (b) If $\mathsf{type} = \mathsf{un\text{-}abort}$: *The simulator can send* $\mathtt{abort}$ *to the trusted party. If it does, we take* $\mathcal{H} = [n] \setminus \mathcal{I}$*.*
4. Trusted party answers parties*:*
   (a) *If the trusted party got* $\mathtt{abort}$ *from the simulator* $\mathcal{S}$*,*
      i. *It sets the abort message* $\mathsf{abortmsg} = \perp$*.*
      ii. *The trusted party sends* y_j *to every party* P_j*,* $j \in [n] \setminus \mathcal{H}$*. The trusted party sends* $\mathsf{abortmsg}$ *to every party* P_j*,* $j \in \mathcal{H}$
   (b) *Otherwise, it sends* y *to every* P_j*,* $j \in [n]$*.*
5. Outputs*: Honest parties always output the message received from the trusted party, while the corrupt parties output nothing. The simulator* $\mathcal{S}$ *outputs an arbitrary function of the initial inputs* $\{x_i\}_{i\in\mathcal{I}}$*, the messages received by the corrupt parties from the trusted party and its auxiliary input.*

Security Definitions. We now define the malicious type security of MPC protocol Π as follows:

Definition 6 (Malicious type security). *Consider* $\mathsf{type} \in \{\mathsf{sa\text{-}abort}, \mathsf{un\text{-}abort}\}$*. Let* $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ *be an* n*-party function. A protocol* Π *t-securely computes the function* f *with malicious* type *security if for every PPT real-world adversary* $\mathcal{A}$ *with auxiliary input* aux*, there exists a PPT simulator* $\mathcal{S}$ *such that for every* $\mathcal{I} \subseteq [n]$ *of size at most* t*, for all* $\boldsymbol{x} \in (\{0,1\}^*)^n$*, for all* $\lambda \in \mathbb{N}$*, it holds that*

$$\mathtt{REAL}_{\Pi,\mathcal{I},\mathcal{A}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda) \stackrel{c}{\equiv} \mathtt{IDEAL}^{\mathsf{type}}_{f,\mathcal{I},\mathcal{S}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$$

3 Impossibility of *BC-BC-P2P* malicious MPC with UA security and black-box use of OT

Before presenting our result, we recall the notion of augmented semi-honest security below, which is used in our argument. Informally, augmented semi-honest security is similar to semi-honest security (where the corrupt parties follow the protocol specifications honestly); with the difference being that in the augmented model, the adversary is allowed to modify its input before the execution begins and abort arbitrarily.

3.1 Augmented semi-honest security model

For the augmented semi-honest adversary, we adopt the definition of *augmented semi-honest security* from [Ode09] and [HL10, p. 28], and we generalize it to n-party protocol cases.

Definition 7 (The augmented semi-honest behavior). *Let Π be a n-party protocol. An* augmented semi-honest behavior (w.r.t., Π) *is a (feasible) strategy that satisfies the following conditions:*

- ***Enter the execution:*** *Depending on its initial input, denoted x, the party may abort before taking any step in the execution of Π. Otherwise, again depending on x, it enters the execution with any input $x' \in \{0,1\}^{|x|}$ of its choice. From this point on, x' is fixed.*
- ***Proper selection of a random-tape:*** *The party selects the random-tape to be used in Π uniformly among all strings of the length specified by Π. That is, the selection of the random-tape is exactly as specified by Π.*
- ***Proper message transmission or abort:*** *In each step of Π, depending on its view of the execution so far, the party may either abort or send a message as instructed by Π. We stress that the message is computed as Π instructs based on input x', the random-tape selected above, and all messages received so far.*
- ***Output:*** *At the end of the interaction, the party produces an output depending on its entire view of the interaction. We stress that the view consists of the initial input x, the random-tape selected above, and all messages received so far.*

Then similar to malicious security, we define augmented semi-honest security by using standard real-world/ideal-world simulation. In the real world, the honest parties faithfully follow the instructions of the protocol, and the corrupted parties behave as described in Definition 7. Following Definition 4, we use $\mathtt{REAL}^{\mathsf{aug\text{-}semi}}_{\Pi,\mathcal{I},\mathcal{A}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$ to denote output of the real-world execution of the augmented semi-honest model. The ideal world is exactly the same as the one in Definition 5.

Security Definitions. We now define the augmented semi-honest security notion of the MPC protocol Π as follows:

Definition 8 (Augmented semi-honest type security). *Consider* $\mathsf{type} \in \{\mathsf{sa\text{-}abort}, \mathsf{un\text{-}abort}\}$. *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an n-party function. A protocol Π t-securely computes the function f with augmented semi-honest* type *security if for every PPT real-world adversary $\mathcal{A}$ who only performs augmented semi-honest behaviors as per Definition 7 with auxiliary input* aux*, there exists a PPT simulator $\mathcal{S}$ such that for every $\mathcal{I} \subseteq [n]$ of size at most t, for all $\boldsymbol{x} \in (\{0,1\}^*)^n$, for all $\lambda \in \mathbb{N}$, it holds that*

$$\mathtt{REAL}^{\mathsf{aug\text{-}semi}}_{\Pi,\mathcal{I},\mathcal{A}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda) \stackrel{c}{\equiv} \mathtt{IDEAL}^{\mathsf{type}}_{f,\mathcal{I},\mathcal{S}(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$$

In this section, we show that a three-round malicious MPC protocol achieving UA in the CRS model with only black-box use of two-round OT protocol, must necessarily use broadcast in the last round (even if broadcast is allowed in the first two rounds). In other words, we show that such a protocol cannot exist in the *BC-BC-P2P* communication pattern.

We refer to Section 1.1 for a high level overview of the proof of our claim, and below we state our formal theorems and proofs.

Theorem 1. *Let f be an efficiently computable five-party function, defined as $f(x_1, x_2, \perp, \perp, \perp) = (y, y, y, y, y)$, where $y = (x_1 \cdot x_2)$, $x_1 \in \{0,1\}$, $x_2 \in \{0,1\}$. Let g be an efficiently computable three-party function, defined as $g(x_1, x_2, \perp) = (y, y, y)$. If there exists a five-party three-round protocol $\Pi = \{(\texttt{frst-msg}_i, \texttt{nxt-msg}_i^k, \texttt{output}_i)\}_{i\in[5],k\in\{2,3\}}$ in the CRS model that securely computes f with malicious* un-abort *security against $t < 5$ corruptions and only uses P2P channels in the last round and makes black-box use of malicious two-round OT, then there exists a three-party two-round protocol $\overline{\Pi}$ that securely computes g, with augmented semi-honest security (as per Definition 8) against $t < 3$ corruptions that makes black-box use of malicious two-round OT protocol.*

The formal proof of the theorem follows.

Step 1: Transformation of Π to four-party two-round solitary MPC protocol Π'. We begin with proving the lemma below.

Lemma 1. *Let f be an efficiently computable five-party function, defined as $f(x_1, x_2, \perp, \perp, \perp) = (y, y, y, y, y)$, where $y = (x_1 \cdot x_2), x_1 \in \{0,1\}, x_2 \in \{0,1\}$. Let f' be an efficiently computable four-party solitary function, defined as $f'(x_1', x_2', \perp, \perp) = (\perp, \perp, \perp, x_1' \cdot x_2')$. If there exists a five-party three-round protocol among $\Pi = \{(\texttt{frst-msg}_i, \texttt{nxt-msg}_i^k, \texttt{output}_i)\}_{i\in[5],k\in\{2,3\}}$ in the CRS model that securely computes f with malicious* un-abort *security against $t < 5$ corruptions and only uses P2P channels in the last round and makes black-box use of malicious two-round OT protocol, then there exists a four-party two-round solitary MPC protocol Π' that securely computes f', with malicious security with abort (as per Definition 6) against $t < 4$ corruptions and makes black-box use of malicious two-round OT protocol.*

Proof. The outline of this proof is as follows: We first describe two scenarios that let us infer that the combined view of $\{P_4, P_5\}$ in Π must suffice to compute the output. Next, we use this inference to describe the transformed protocol Π' and argue its security.

Consider Scenario 1 (Figure 3.1) as an execution of Π, where the adversary corrupts $\{P_1, P_2, P_3\}$. Recall that by assumption Π has the communication pattern *BC-BC-P2P*. The adversary's strategy is to behave honestly, except that the last round *P2P* messages to the honest parties are dropped.

Figure 3.1: Scenario 1

Adversarial setting: P_1, P_2, P_3 are corrupted, while P_4, P_5 are honest.
Private inputs: P_1 has a private input $x_1 \in \{0,1\}$. P_2 has a private input $x_2 \in \{0,1\}$. Party P_3, P_4, P_5 have input $x_3, x_4, x_5 \leftarrow \perp$.
CRS: Set up the common reference strings $\mathsf{crs} \xleftarrow{\$} \texttt{setup}(1^\lambda)$.

First round (BC):
Every party P_i ($i \in [5]$) computes $(\mathsf{msg}_i^1, \{\mathsf{msg}_{i\to j}^1\}_{j\in[5]}) \leftarrow \mathtt{frst\text{-}msg}_i(\mathsf{crs}, x_i)$, and sends the message msg_i^1 over BC channel and $\mathsf{msg}_{i\to j}^1$ over $P2P$ channels to P_j for $j \in [5]$.

Second round (BC):
Every party P_i ($i \in [5]$) computes $(\mathsf{msg}_i^2, \{\mathsf{msg}_{i\to j}^2\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}_i^2(\mathsf{crs}, x_i, \{\mathsf{msg}_j^1, \mathsf{msg}_{j\to i}^1\}_{j\in[5]})$, and msg_i^2 over BC channel and sends $\mathsf{msg}_{i\to j}^2$ over $P2P$ channels to P_j for $j \in [5]$.

Third round (P2P):

1. Every party P_i ($i \in [5]$) computes $(\{\mathsf{msg}_{i\to j}^3\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}_i^3(\mathsf{crs}, x_i, \{\mathsf{msg}_j^k, \mathsf{msg}_{j\to i}^k\}_{j\in[5],k\in\{1,2\}})$.
2. For $i \in \{4, 5\}$, P_i sends $\mathsf{msg}_{i\to j}^3$ to P_j for $j \in [5]$.

Next, we describe another case, namely Scenario 2 in Figure 3.2, where the adversary corrupting P_1, P_2, P_3 behaves honestly except that it drops the last round $P2P$ message only to one of the two honest parties, namely P_5.

Figure 3.2: Scenario 2

Adversarial setting: P_1, P_2, P_3 are corrupted, while P_4, P_5 are honest.
Private inputs: P_1 has a private input $x_1 \in \{0, 1\}$. P_2 has a private input $x_2 \in \{0, 1\}$. Party P_3, P_4, P_5 have input $x_3, x_4, x_5 \leftarrow \perp$.
CRS: Set up the common reference strings $\mathsf{crs} \xleftarrow{\$} \mathtt{setup}(1^\lambda)$.

First round (BC):
Every party P_i ($i \in [5]$) computes $(\mathsf{msg}_i^1, \{\mathsf{msg}_{i\to j}^1\}_{j\in[5]}) \leftarrow \mathtt{frst\text{-}msg}_i(\mathsf{crs}, x_i)$, and sends the message msg_i^1 over BC channel and $\mathsf{msg}_{i\to j}^1$ over $P2P$ channels to P_j for $j \in [5]$.

Second round (BC):
Every party P_i ($i \in [5]$) computes $(\mathsf{msg}_i^2, \{\mathsf{msg}_{i\to j}^2\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}_i^2(\mathsf{crs}, x_i, \{\mathsf{msg}_j^1, \mathsf{msg}_{j\to i}^1\}_{j\in[5]})$, and sends msg_i^2 over BC channel and $\mathsf{msg}_{i\to j}^2$ over $P2P$ channels to P_j for $j \in [5]$.

Third round (P2P):

1. Every party P_i ($i \in [5]$) computes $(\{\mathsf{msg}_{i\to j}^3\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}_i^3(\mathsf{crs}, x_i, \{\mathsf{msg}_j^k, \mathsf{msg}_{j\to i}^k\}_{j\in[5],k\in\{1,2\}})$.
2. For $i \in \{1, 2, 3\}$, P_i sends $\mathsf{msg}_{i\to 4}^3$ to P_4.
3. For $i \in \{4, 5\}$, P_i sends $\mathsf{msg}_{i\to j}^3$ to P_j for $j \in [5]$.

Intutively, in Scenario 2 (in Figure 3.2), P_4 receives messages from all parties as per an execution where everyone behaved honestly. This is because P_4 is oblivious of what occurred on the $P2P$ channels towards other parties in the last round. More formally, the view of P_4 in Scenario 2 is identically distributed to its view in an honest execution of Π. Correctness dictates that an honest execution of Π must lead to P_4 outputting $y = x_1 \cdot x_2$ with overwhelming probability. We can thus infer that P_4 must output y in Scenario 2 with overwhelming probability. Next, it follows from malicious $\mathsf{un\text{-}abort}$ security of Π that P_5 must also output y in Scenario 2.

Next, we argue that Scenario 1 must lead to honest parties outputting y as well. Note that the view of P_5 is identically distributed in Scenarios 1 and 2. Therefore, one can infer that P_5 must output y in Scenario 1. It now follows from unanimity that P_4 must compute y in Scenario 1 with overwhelming probability as well. Specifically, this means that the combined view of $\{P_4, P_5\}$ at the end of Round 2 itself suffices to compute the output, as the only communication P_4 and P_5 receive in Round 3 of Scenario 1 is messages from each other, which is a function of their view at the end of Round 2 itself.

We are now ready to present the transformed *two-round solitary* protocol Π' computing $f'(x'_1, x'_2, \bot, \bot) = (\bot, \bot, \bot, x'_1 \cdot x'_2)$. In Π', each P'_i $(i \in [3])$ the role of P_i in Π for the first two rounds. P'_4 emulates the role of both $\{P_4, P_5\}$ in Π in the first two rounds. The formal description of the protocol appears below in Figure 3.3.

Figure 3.3: The four-party two-round solitary protocol Π'

Primitives: A five-party three-round protocol $\Pi = \{(\mathtt{frst\text{-}msg}_i, \mathtt{nxt\text{-}msg}_i^2, \mathtt{nxt\text{-}msg}_i^3, \mathtt{output}_i)\}_{i\in[5]}$ that securely computes f with malicious $\mathsf{un\text{-}abort}$ security against $t < 5$ corruptions, where all the parties obtain the output, and only use $P2P$ channels in the last round (first two rounds use BC channels as well).
Private inputs: P'_1 has a private input $x'_1 \in \{0,1\}$. P'_2 has a private input $x'_2 \in \{0,1\}$. Party P'_3 and P'_4 have input $x'_3, x'_4 \leftarrow \bot$. Note that P'_4 is the only party that obtains the output.
CRS: Set up the common reference strings $\mathsf{crs} \xleftarrow{\$} \mathtt{setup}(1^\lambda)$.

First round (BC):

- For $i \in [3]$, P'_i computes $(\mathsf{msg}_i^1, \{\mathsf{msg}_{i\to j}^1\}_{j\in[5]}) \leftarrow \mathtt{frst\text{-}msg}_i(\mathsf{crs}, x'_i)$, and sends the message msg_i^1 over BC channel, $\mathsf{msg}_{i\to j}^1$ over $P2P$ channels to P'_j for $j \in [4]$ and $\mathsf{msg}_{i\to 5}^1$ to P'_4.
- P'_4 sets $x'_5 \leftarrow \bot$, and computes $(\mathsf{msg}_i^1, \{\mathsf{msg}_{i\to j}^1\}_{j\in[5]}) \leftarrow \mathtt{frst\text{-}msg}_i(\mathsf{crs}, x'_i)$, where $i \in \{4,5\}$, and sends $(\mathsf{msg}_4^1, \mathsf{msg}_5^1)$ over BC channel and $(\mathsf{msg}_{4\to j}^1, \mathsf{msg}_{5\to j}^1)$ over $P2P$ channels to P'_j for $j \in [4]$.

Second round (BC):

– For $i \in [3]$, P'_i computes $(\mathsf{msg}^2_i, \{\mathsf{msg}^2_{i\to j}\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}^2_i(\mathsf{crs}, x'_i, \{\mathsf{msg}^1_j, \mathsf{msg}^1_{j\to i}\}_{j\in[5]})$, and sends the message msg^2_i over BC channel, $\mathsf{msg}^2_{i\to j}$ over $P2P$ channels to P'_j for $j \in [4]$ and $\mathsf{msg}^2_{i\to 5}$ over $P2P$ channels to P'_4.
– P'_4 computes $(\mathsf{msg}^2_i, \{\mathsf{msg}^2_{i\to j}\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}^2_i(\mathsf{crs}, x'_i, \{\mathsf{msg}^1_j, \mathsf{msg}^1_{j\to i}\}_{j\in[5]})$, where $i \in \{4, 5\}$, and sends $(\mathsf{msg}^2_4, \mathsf{msg}^2_5)$ over BC channel and $(\mathsf{msg}^2_{4\to j}, \mathsf{msg}^2_{5\to j})$ over $P2P$ channels to P'_j for $j \in [4]$.

Output Computation:
The output obtaining party P'_4 does the following:

1. For $i \in \{4, 5\}$, compute $(\{\mathsf{msg}^3_{i\to j}\}_{j\in[5]}) \leftarrow \mathtt{nxt\text{-}msg}^3_i(\mathsf{crs}, x'_i, \{\mathsf{msg}^k_j, \mathsf{msg}^k_{j\to i}\}_{j\in[5],k\in[2]})$.
2. P'_4 outputs $y \leftarrow \mathtt{output}_4(\mathsf{crs}, x'_4, \{\mathsf{msg}^k_j, \mathsf{msg}^k_{j\to 4}\}_{j\in[5],k\in[2]}, \mathsf{msg}^3_{4\to 4}, \mathsf{msg}^3_{5\to 4})$.

We claim that Π' securely computes f'.

– **Correctness:** Consider an execution of Π' where everyone behaves honestly. Note that the view of P'_4 at the end of Π' is identical to the combined view of $\{P_4, P_5\}$ at the end of Round 2 of Π. Based on the conclusion above, this view must suffice for the correct output computation of f' by P'_4 (who is the output obtaining party in Π'); thereby correctness holds.

– **Security:** Informally, security follows from the fact that honest parties in Π' essentially send the same messages as in Π, except that fewer messages are sent. Therefore, security of Π' must follow intuitively based on the security of Π (which is assumed to be secure with malicious $\mathsf{un\text{-}abort}$ security). More formally, we transform an adversary $\mathcal{A}_{\Pi'}$ attacking Π' into an admissible adversary $\mathcal{M}_{\mathsf{intf}}$ of Π (we need to do that since the simulators $\mathcal{S}_{\mathcal{I}}$ only work against adversaries attacking the protocol Π, where $\mathcal{I}$ is the set of indices of corrupted parties in Π). $\mathcal{M}_{\mathsf{intf}}$ runs internally $\mathcal{A}_{\Pi'}$ and acts as a proxy for the messages between the simulator $\mathcal{S}_{\mathcal{I}}$ and $\mathcal{A}_{\Pi'}$, withholding the third-round messages that honest parties are not supposed to send in Π'. The Figure 3.4 formally describes $\mathcal{M}_{\mathsf{intf}}$. In this, we denote as the *left interface*, the interface where the adversary sends and receives the protocol messages.

Figure 3.4: The adversary $\mathcal{M}_{\mathsf{intf}}$

Notation: Let $\mathbb{H}'$ be the set of indices of the honest parties and $\mathcal{I}'$ be the indices of the corrupted parties in Π'. $\mathcal{M}_{\mathsf{intf}}$ internally runs the adversary $\mathcal{A}_{\Pi'}$, and is equipped with a left interface, where it receives the messages computed on behalf of the honest parties and sends the messages computed on the behalf of the corrupted parties.

k-th round (BC) ($\mathbf{k} \in [\mathbf{2}]$):

1. If $4 \in \mathcal{I}'$, then $\mathbb{H} \leftarrow \mathbb{H}'$.
   (a) Upon receiving $(\mathsf{msg}_h^k, \{\mathsf{msg}_{h\to i}^k\}_{i\in\mathcal{I}'}, \mathsf{msg}_{h\to 5}^k)$ on the left interface with $h \in \mathbb{H}$, $\mathcal{M}_{\mathsf{intf}}$ parses the message msg_h^k in Π as the message $\widetilde{\mathsf{msg}}_h^k$ in Π', parses $\{\mathsf{msg}_{h\to i}^k\}_{i\in\mathcal{I}'\setminus\{4\}}$ as $\{\widetilde{\mathsf{msg}}_{h\to i}^k\}_{i\in\mathcal{I}'\setminus\{4\}}$, parses $(\mathsf{msg}_{h\to 4}^k, \mathsf{msg}_{h\to 5}^k)$ as $\widetilde{\mathsf{msg}}_{h\to 4}^k$ and forwards these messages to $\mathcal{A}_{\Pi'}$ in Π'.
   (b) Upon receiving the messages $(\{\widetilde{\mathsf{msg}}_i^k\}_{i\in\mathcal{I}'}, \{\widetilde{\mathsf{msg}}_{i\to h}^k\}_{i\in\mathcal{I}',h\in\mathbb{H}'})$ sent by $\mathcal{A}_{\Pi'}$, $\mathcal{M}_{\mathsf{intf}}$ parses $\{\widetilde{\mathsf{msg}}_i^k\}_{i\in\mathcal{I}'\setminus\{4\}}$ as $\{\mathsf{msg}_i^k\}_{i\in\mathcal{I}'\setminus\{4\}}$, parses $\widetilde{\mathsf{msg}}_4^k$ as $(\mathsf{msg}_4^k, \mathsf{msg}_5^k)$, parses $\{\widetilde{\mathsf{msg}}_{i\to h}^k\}_{i\in\mathcal{I}'\setminus\{4\},h\in\mathbb{H}'}$ as $\{\mathsf{msg}_{i\to h}^k\}_{i\in\mathcal{I}'\setminus\{4\},h\in\mathbb{H}'}$, parses $\{\widetilde{\mathsf{msg}}_{4\to h}^k\}_{h\in\mathbb{H}'}$ as $\{\mathsf{msg}_{4\to h}^k, \mathsf{msg}_{5\to h}^k\}_{h\in\mathbb{H}'}$, and forwards them to the left interface, where it is acting as corrupted parties for Π. Here we note that these messages could be invalid messages or just no message at all, but it is still a valid attack for Π.
2. If $4 \in \mathbb{H}'$, then $\mathbb{H} \leftarrow \mathbb{H}' \cup \{5\}$.
   (a) Upon receiving $(\mathsf{msg}_h^k, \{\mathsf{msg}_{h\to i}^k\}_{i\in\mathcal{I}'})$ on the left interface with $h \in \mathbb{H}$, $\mathcal{M}_{\mathsf{intf}}$ parses the message $\{\mathsf{msg}_h^k\}_{h\in\mathbb{H}\setminus\{4,5\}}$ in Π as the message $\{\widetilde{\mathsf{msg}}_h^k\}_{h\in\mathbb{H}\setminus\{4,5\}}$ in Π', parses $(\mathsf{msg}_4^k, \mathsf{msg}_5^k)$ as $\widetilde{\mathsf{msg}}_4^k$, parses $\{\mathsf{msg}_{h\to i}^k\}_{h\in\mathbb{H}\setminus\{4,5\},i\in\mathcal{I}'}$ as $\{\widetilde{\mathsf{msg}}_{h\to i}^k\}_{h\in\mathbb{H}\setminus\{4,5\},i\in\mathcal{I}'}$, parses $\{\mathsf{msg}_{4\to i}^k, \mathsf{msg}_{5\to i}^k\}_{i\in\mathcal{I}'}$ as $\{\widetilde{\mathsf{msg}}_{4\to i}^k\}_{i\in\mathcal{I}'}$, and forwards the message to $\mathcal{A}_{\Pi'}$ in Π'.
   (b) Upon receiving the messages $(\{\widetilde{\mathsf{msg}}_i^k\}_{i\in\mathcal{I}'}, \{\widetilde{\mathsf{msg}}_{i\to h}^k\}_{i\in\mathcal{I}',h\in\mathbb{H}'})$ sent by $\mathcal{A}_{\Pi'}$, $\mathcal{M}_{\mathsf{intf}}$ parses $\{\widetilde{\mathsf{msg}}_i^k\}_{i\in\mathcal{I}'}$ as $\{\mathsf{msg}_i^k\}_{i\in\mathcal{I}'}$, parses $\{\widetilde{\mathsf{msg}}_{i\to h}^k\}_{i\in\mathcal{I}',h\in\mathbb{H}'\setminus\{4\}}$ as $\{\mathsf{msg}_{i\to h}^k\}_{i\in\mathcal{I}',h\in\mathbb{H}'\setminus\{4\}}$, parses $\{\widetilde{\mathsf{msg}}_{i\to 4}^k\}_{i\in\mathcal{I}'}$ as $\{\mathsf{msg}_{i\to 4}^k, \mathsf{msg}_{i\to 5}^k\}_{i\in\mathcal{I}'}$, and forwards them to the left interface, where it is acting as corrupted parties for Π.

We are now ready to show how the simulator $\mathcal{S}'_{\mathcal{I}'}$ of Π' for the case where $\mathcal{I}'$ are the set of indices of corrupted parties. The simulator $\mathcal{S}'_{\mathcal{I}'}$ is formally described in Figure 3.5.

Figure 3.5: $\mathcal{S}'_{\mathcal{I}'}$

$\mathcal{S}'_{\mathcal{I}'}$ performs the following steps:

- If $4 \in \mathcal{I}'$, $\mathcal{I} \leftarrow \mathcal{I}' \cup \{5\}$. Otherwise, $\mathcal{I} \leftarrow \mathcal{I}'$.
- Invoke $\mathcal{S}_{\mathcal{I}}$ for the adversary $\mathcal{M}_{\mathsf{intf}}$, querying and receiving responses to and from its left interface.
- Work as a proxy between the ideal functionality and $\mathcal{S}_{\mathcal{I}}$.
- If $\mathcal{S}_{\mathcal{I}}$ send third round messages, ignore them.

In the end, $\mathcal{S}'_{\mathcal{I}'}$ outputs whatever $\mathcal{S}_{\mathcal{I}}$ outputs, and halts.

If an adversary $\mathcal{A}_{\Pi'}$ is able to distinguish when the messages are produced by $\mathcal{S}'_{\mathcal{I}'}$ from the case when the messages are generated from honest parties running Π', then we can show an adversary $\mathcal{A}_{\Pi}$ that contradicts the security of Π. $\mathcal{A}_{\Pi}$ internally runs $\mathcal{M}_{\mathsf{intf}}$ in the reduction, and $\mathcal{M}_{\mathsf{intf}}$ will internally run $\mathcal{A}_{\Pi'}$.

Step 2: Transformation of Π' to three-party two-round protocol $\overline{\Pi}$ with augmented semi-honest security. We argue this step via the lemma below.

Lemma 2. *Let f' be an efficiently computable four-party function, defined as $f'(x'_1, x'_2, \perp, \perp) = (\perp, \perp, \perp, x'_1 \cdot x'_2)$, where $x'_1 \in \{0,1\}, x'_2 \in \{0,1\}$. Let g be an efficiently computable three-party function, defined as $g(\overline{x}_1, \overline{x}_2, \perp) = (y, y, y)$, where $y = \overline{x}_1 \cdot \overline{x}_2$. If there exists a four-party two-round solitary protocol $\Pi' = \{(\texttt{frst-msg}_i, \texttt{nxt-msg}_i^2), \texttt{output}\}_{i\in[4]}$ in the CRS model that securely computes f' with abort security against $t < 4$ corruptions and makes black-box use of malicious two-round OT, then there exists a three-party two-round protocol $\overline{\Pi}$ that securely computes g, with augmented semi-honest security (as per Definition 8) against $t < 3$ corruptions that makes black-box use of a two-round malicious OT.*

Proof. We construct a two-round augmented semi-honest protocol $\overline{\Pi}$ with party $\{\overline{P}_1, \overline{P}_2, \overline{P}_3\}$, in Figure 3.6. The idea is to follow the standard player emulation technique: In $\overline{\Pi}$, each party $\overline{P}_i$ ($i \in [3]$) plays the role of the analogous party P'_i in Π'. Further, a designated party $\overline{P}_1$ in $\overline{\Pi}$ emulates the role of P'_4 in Π' by publicly choosing randomness and computing messages on behalf of P'_4 (also used in [ABG+20, Lemma 4.10]).

Figure 3.6: The three-party two-round augmented semi-honest protocol $\overline{\Pi}$

Primitives: A four-party two-round augmented semi-honest solitary protocol $\Pi' = (\{(\texttt{frst-msg}_i, \texttt{nxt-msg}_i^2)\}_{i\in[4]}, \texttt{output})$ computing f'. Here we note that a malicious secure solitary MPC implies an augmented semi-honest secure (Definition 8) solitary MPC [HL10, Proposition 2.3.5].
Private inputs: $\overline{P}_1$ has a private input $\overline{x}_1 \in \{0,1\}$. $\overline{P}_2$ has a private input $\overline{x}_2 \in \{0,1\}$. Party $\overline{P}_3$ has the input $\overline{x}_3 \leftarrow \perp$.
CRS: Set up the common reference strings $\mathsf{crs} \xleftarrow{\$} \texttt{setup}(1^\lambda)$.

First round (BC):

- Every party $\overline{P}_i$ ($i \in [3]$) computes $(\mathsf{msg}_i^1, \{\mathsf{msg}_{i\to j}^1\}_{j\in[4]}) \leftarrow \texttt{frst-msg}_i$ $(\mathsf{crs}, \overline{x}_i)$ and sends the message msg_i^1 over BC channel and $\mathsf{msg}_{i\to j}^1$ over $P2P$ channels to $\overline{P}_j$ for ($j \in [3]$), and $\mathsf{msg}_{i\to 4}^1$ over $P2P$ channels to $\overline{P}_1$.

– $\overline{P}_1$ samples the randomness r' in the same way as how P'_4 sample randomness in Π', sets $x'_4 \leftarrow \perp$, computes $(\mathsf{msg}^1_4, \{\mathsf{msg}^1_{4\to j}\}_{j\in[4]}) \leftarrow \mathtt{frst\text{-}msg}_4(\mathsf{crs}, x'_4; r')$, and on behalf of P'_4 in Π' sends messages msg^1_4 over BC channel, and $\{\mathsf{msg}^1_{4\to j}\}_{j\in[3]}$ to $\overline{P}_j$ over $P2P$ channels.

Second round (BC):

– Every party $\overline{P}_i$ ($i \in [3]$) computes $(\mathsf{msg}^2_i, \{\mathsf{msg}^2_{i\to j}\}_{j\in[4]}) \leftarrow \mathtt{nxt\text{-}msg}_i(\mathsf{crs}, \overline{x}_i, \{\mathsf{msg}^1_j, \mathsf{msg}^1_{j\to i}\}_{j\in[4]})$ and sends the message msg^2_i over BC channel and $\mathsf{msg}^2_{i\to j}$ over $P2P$ channels to $\overline{P}_j$ for ($j \in [3]$), and $\mathsf{msg}^2_{i\to 4}$ over $P2P$ channels to $\overline{P}_1$.
– $\overline{P}_1$ computes $(\mathsf{msg}^2_4, \{\mathsf{msg}^2_{4\to j}\}_{j\in[4]}) \leftarrow \mathtt{nxt\text{-}msg}_4(\mathsf{crs}, x'_4, \{\mathsf{msg}^1_j, \mathsf{msg}^1_{j\to 4}\}_{j\in[4]}; r')$, and sends $(r', \{\mathsf{msg}^1_{j\to 4}\}_{j\in[4]}, \mathsf{msg}^2_4, \{\mathsf{msg}^2_{j\to 4}\}_{j\in[4]})$ over BC channel.

Output Computation:

– Every party $\overline{P}_i$ ($i \in [3]$) sets $x'_4 \leftarrow \perp$, runs $\mathtt{output}(\mathsf{crs}, x'_4, \{\mathsf{msg}^k_j, \mathsf{msg}^k_{j\to 4}\}_{k\in[2],j\in[4]}; r')$ to obtain the output y.

We argue security of $\overline{\Pi}$ as follows: For every subset $\mathcal{I} \subseteq [3]$, let $\overline{\mathcal{S}}_{\mathcal{I}}$ be the simulator when the subset $\mathcal{I}$ are corrupted, then we set $\overline{\mathcal{S}}_{\mathcal{I}} = \mathcal{S}'_{\mathcal{I}\cup\{4\}}$; therefore the security guarantee of $\overline{\Pi}$ follows directly from the security of Π'.

To arrive at our final contradiction, we use the following impossibility result of [ABG+20, Theorem 1.4].

Theorem 2. *Consider the three-party functionality f, defined as $f(x_1, x_2, \perp) = (y, y, y)$, where $y = (x_1 \cdot x_2)$, $x_1 \in \{0,1\}$, $x_2 \in \{0,1\}$. Then f cannot be securely realized by a two-round semi-honest protocol against $t < 3$ corruptions in the CRS model making a black-box use of two-round OT protocol.*

We are now ready to state and prove our impossibility result.

Theorem 3. *There exists five-party function f such that no three-round protocol in the CRS model, that makes only black-box use of malicious two-round OT protocol, can compute f with malicious* un-abort *security against $t < 5$ corruptions, such that the first two rounds use BC and $P2P$ channels and the last round uses only $P2P$ channel.*

Proof. Let f be $f(x_1, x_2, \perp, \perp, \perp) = (y, y, y, y, y)$, where $y = (x_1 \cdot x_2)$, $x_1 \in \{0,1\}$, $x_2 \in \{0,1\}$. Then by applying Theorem 1, we have the three-party two-round protocol $\overline{\Pi}$ that securely computes $g(x_1, x_2, \perp) = (y, y, y)$, with augmented semi-honest security. However, this contradicts the impossibility of Theorem 2 which states that such a construction $\overline{\Pi}$ cannot exist; concluding our proof.

Finally, we note that even though the impossibility of [ABG+20] focuses on semi-honest security, it can be extended to augmented semi-honest security. More precisely, followed by the observation in [IKSS22b], while the main theorem statement of [ABG+20] only refers to a separation between two-round MPC and two-round *semi-honest* OT, the impossibility still holds even if the oracle is replaced by a *malicious* secure two-round OT. Since augmented semi-honest security is strictly weaker than malicious security, we can claim that the same impossibility holds also when the oracle is replaced by a two-round augmented semi-honest OT protocol.

4 Black-Box Broadcast-Optimal Two-Round Compiler

4.1 P2P-P2P, SA, Correlated Randomness Model, $t < n$

Our compiler takes as input a two-round actively secure MPC protocol Π_{bc}, that tolerates $t < n$ corruptions and uses a broadcast channel in both rounds. It transforms Π_{bc} into a protocol over peer-to-peer channels instead. Importantly, this transformation preserves the black-box (BB) nature of the original protocol's use of cryptographic primitives. Thus, if Π_{bc} makes BB use of these primitives, the resulting protocol inherits this property. Previous compilers, such as [CGZ20] relied on non-black-box use of the underlying primitives.

Informally, our compiler works as follows: In the first round, every party P_i samples a uniformly random value k_i, which will serve as the key of the one-time pad encryption. Then, P_i defines a constant function f_i that takes as input the Round 1 messages that P_i would have broadcasted in Π_{bc} and outputs k_i. Then in addition to sending the first-round messages of Π_{bc} over a peer-to-peer channel, the parties send a garbled circuit which computes the function f_i and additively share labels of this garbled circuit.

In the second round, each party P_i computes a one-time pad encryption on her second-round messages of Π_{bc} using the key k_i. Then, P_i sends her an encrypted version of the second-round messages and for each first-round messages of Π_{bc} received privately she sends the appropriate shares corresponding to the label in everyone else's garbled circuit, and echos the received garbled circuits in the first round.

This mechanism ensures that parties can access the second-round messages of Π_{bc} only if they agree on the version of the first-round messages of Π_{bc} they received during the first round. If any party sends inconsistent first-round messages of Π_{bc}, the one-time pad key required to reveal the second-round messages remains inaccessible due to the partial evaluation resilience of the garbled circuit. As a consequence, the second round of Π_{bc} remains hidden.

The compiler is formally described in Figure 4.1.

Figure 4.1: The black-box broadcast-optimal two-round compiler Π_{Comp}

Primitives: A n-party two-round protocol $\Pi_{\mathsf{bc}} = \{(\mathtt{frst\text{-}msg}_i, \mathtt{nxt\text{-}msg}_i^2, \mathtt{output}_i)\}_{i\in[n]}$ that securely computes f with malicious $\mathsf{un\text{-}abort}$ security against $t < n$ corruptions, with every round of communication over BC channel, and a garbling scheme $(\mathtt{garble}, \mathtt{eval})$.
Private input: Every party P_i has a private input $x_i \in \{0,1\}^*$.
Correlated randomness: The correlated randomness $(r_1, \ldots, r_n) \leftarrow \mathcal{D}_{\mathsf{corr}}^{\Pi_{\mathsf{bc}}}$ (where $\mathcal{D}_{\mathsf{corr}}^{\Pi_{\mathsf{bc}}}$ is the predefined efficiently sampleable distribution for sampling correlated randomness in Π_{bc}) is sampled at the onset of the protocol, and every party P_i receives r_i.
Notations: For simplicity assume that each round message has the same length and it is ℓ bits long, so each circuit has $\mathsf{L} = n \cdot \ell$ input bits.

First round (P2P): Every party P_i does the following:

1. Let $\mathsf{msg}_i^1 \leftarrow \mathtt{frst\text{-}msg}_i(x_i, r_i)$ be P_i's first-round messages in Π_{bc}.
2. Sample $\mathsf{k}_i \xleftarrow{\$} \{0,1\}^\ell$. Set f_i a constant function that take as input first-round messages $\{\mathsf{msg}_k^1\}_{k\in[n]}$, and output k_i. Then set $\mathtt{C}_i$ the boolean circuit that achieves f_i.
3. Compute $(\mathtt{GC}_i, \mathbf{K}_i) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C}_i)$, where $\mathbf{K}_i = \{\mathbf{K}_{i,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}}$.
4. For every $\alpha \in [\mathsf{L}]$ and $b \in \{0,1\}$, sample n uniform random strings $\{\mathbf{K}_{i\to k,\alpha}^b\}_{k\in[n]}$, such that $\mathbf{K}_{i,\alpha}^b = \bigoplus_{k\in[n]} \mathbf{K}_{i\to k,\alpha}^b$.
5. Send to every party P_j the message $(\mathtt{GC}_i, \mathsf{msg}_i^1, \{\mathbf{K}_{i\to j,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$.

Second round (P2P): Every party P_i does the following:

1. If P_i does not receive a message from some other party (or an abort message), she aborts.
2. Otherwise, let $(\mathtt{GC}_j, \mathsf{msg}_{j\to i}^1, \{\mathbf{K}_{j\to i,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$ be the first-round messages received from P_j.
3. Compute $\mathsf{msg}_i^2 \leftarrow \mathtt{nxt\text{-}msg}_i^2(x_i, \{\mathsf{msg}_{k\to i}^1\}_{k\in[n]}, r_i)$, and compute $\overline{\mathsf{msg}}_i^2 \leftarrow \mathsf{k}_i \oplus \mathsf{msg}_i^2$.
4. Concatenate all received messages $\{\mathsf{msg}_{k\to i}^1\}_{k\in[n]}$ as $(\mu_{i,1}, \ldots, \mu_{i,\mathsf{L}}) \leftarrow (\mathsf{msg}_{1\to i}^1, \ldots, \mathsf{msg}_{n\to i}^1) \in \{0,1\}^{\mathsf{L}}$.
5. Let $\overline{\mathsf{GC}}_i$ be the set of garbled circuits received from the other parties in the first round.
6. Send to all parties the message $(\overline{\mathsf{GC}}_i, \overline{\mathsf{msg}}_i^2, \{\mathbf{K}_{k\to i,\alpha}^{\mu_{i,\alpha}}\}_{k\in[n],\alpha\in[\mathsf{L}]})$.

Output Computation: Every party P_i does the following:

1. If P_i does not receive a message from some other party (or receives an abort message), she aborts; Otherwise, let $(\overline{\mathsf{GC}}_j, \widetilde{\mathsf{msg}}_{j\to i}^2, \{\widetilde{\mathbf{K}}_{1\to j,\alpha}\}_{\alpha\in[\mathsf{L}]}, \ldots, \{\widetilde{\mathbf{K}}_{n\to j,\alpha}\}_{\alpha\in[\mathsf{L}]})$ be the second-round messages received from party P_j.

2. Check if the set of garbled circuits $\{\overline{\mathsf{GC}}_k\}_{k\in[n]}$ echoed in second round are consistent with the garbled circuits received in first round. If this is not the case, abort.
3. For every $j \in [n]$ and $\alpha \in [\mathsf{L}]$, reconstruct each garbled label by computing $\widetilde{\mathbf{K}}_{j,\alpha} \leftarrow \bigoplus_{k\in[n]} \widetilde{\mathbf{K}}_{j\to k,\alpha}$.
4. For every $j \in [n]$, evaluate the garbled circuits as $\mathsf{k}_j \leftarrow \mathtt{eval}(\mathtt{GC}_j, \{\widetilde{\mathbf{K}}_{j,\alpha}\}_{\alpha\in[\mathsf{L}]})$. If any evaluation fails, aborts. Otherwise, compute $\mathsf{msg}^2_{j\to i} \leftarrow \widetilde{\mathsf{msg}}^2_{j\to i} \oplus \mathsf{k}_j$.
5. Compute and output $y \leftarrow \mathtt{output}_i(x_i, \{\mathsf{msg}^1_{k\to i}\}_{k\in[n]}, \{\mathsf{msg}^2_{k\to i}\}_{k\in[n]}; r_i)$.

Then we have the following theorem.

Theorem 4 (P2P-P2P, SA, t < n). *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Let Π_{bc} be a BC-BC protocol that securely computes f with malicious* un-abort *security against $t < n$ corruptions with the additional constraint that a simulator straight-line extract the inputs before the last round. Then, assuming the existence of secure garbling schemes (Definition 1), the protocol Π_{Comp} from Figure 4.1 can compute f with malicious* sa-abort *security that uses only P2P channels against $t < n$ corruptions, and the protocol only makes a black-box use of the underlying MPC protocol Π_{bc} and the garbling scheme.*

Proof. We prove the protocol Π_{Comp} from Figure 4.1 computes f securely with selective abort. The proof follows a similar approach to the one of [CGZ20], with some modifications given that our compiler makes black-box use of the underlying cryptographic primitives.

Let $\mathcal{A}$ be the adversary attacking protocol Π_{Comp}, and let $\mathcal{I} \subseteq [n]$ be the set of parties corrupted by $\mathcal{A}$ and $\mathbb{H}$ the set of honest parties. We assume that $\mathcal{A}$ is deterministic, and its output consists of its view during the protocol (i.e. the auxiliary input, the input and correlated randomness of all corrupted parties, and all the messages it sends and receives). For every honest party P_j, we define a receiver-specific adversary $\mathcal{A}_j$ for the protocol Π_{bc}, and we describe it in Figure 4.2.

Figure 4.2: The Receiver Specific Adversary $\mathcal{A}_j$

Notation: To simplify the notation we assume that if $\mathcal{A}_j$ sends an abort message so does $\mathcal{A}$ but we will not write it explicitly in the rest of the description of $\mathcal{A}_j$.

$\mathcal{A}_j$ starts by invoking $\mathcal{A}$ on the input $\{x_i\}_{i\in\mathcal{I}}$, the correlated randomness $\{r_i\}_{i\in\mathcal{I}}$, and auxiliary information aux. Then $\mathcal{A}_j$ does the follows:

First round:

- Upon receiving the first-broadcast-round message msg_h^1 from a honest party P_h in Π_{bc}, $\mathcal{A}_j$ samples a uniformly random string $\mathbf{K}_{h\to i,\alpha}^b$ for every $i \in \mathcal{I}$, every $\alpha \in [\mathsf{L}]$ and every $b \in \{0,1\}$. $\mathcal{A}_j$ computes $\mathsf{k}_h \xleftarrow{\$} \{0,1\}^\ell$, set f_h a constant function that take as input first-round messages $\{\mathsf{msg}_k^1\}_{k\in[n]}$, and output k_h. Then set $\mathtt{C}_h$ the boolean circuit that achieves f_h. $\mathcal{A}_j$ computes $\mathtt{GC}_h \leftarrow \mathtt{simGC}(1^\lambda, \phi(\mathtt{C}_h), \text{“}ckt\text{”})$. Then, $\mathcal{A}_j$ sends $(\mathtt{GC}_h, \mathsf{msg}_h^1, \{\mathbf{K}_{h\to i,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$ to $\mathcal{A}$, on behalf of the honest party P_h, sends to each corrupted party P_i over $P2P$ channels in Π_{Comp}.
- Upon receiving $(\mathtt{GC}_i, \mathsf{msg}_{i\to h}^1, \{\mathbf{K}_{i\to h,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$ from $\mathcal{A}$ (on behalf of every corrupted party P_i to every honest party P_h in Π_{Comp}) $\mathcal{A}_j$ checks if there exists $i \in \mathcal{I}$ and $h, h' \in \mathbb{H}$ s.t $\mathsf{msg}_{i\to h}^1 \neq \mathsf{msg}_{i\to h'}^1$ then $\mathcal{A}_j$ aborts, otherwise $\mathcal{A}_j$ broadcasts the message $\mathsf{msg}_{i\to j}^1$ for every corrupted party P_i in protocol Π_{bc}.

Second round:

- Upon receiving the second-broadcast-round message msg_h^2 from a honest party P_h in Π_{bc}, $\mathcal{A}_j$ invokes the garbling scheme simulator to obtain $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \mathtt{simGC}(1^\lambda, \mathsf{k}_h, \text{“}input\text{”})$. $\mathcal{A}_j$ computes $\overline{\mathsf{msg}}_h^2 \leftarrow \mathsf{k}_h \oplus \mathsf{msg}_h^2$.
- Concatenate all received messages $\{\mathsf{msg}_{k\to h}^1\}_{k\in[n]}$ as: $(\mu_{h,1}, \ldots, \mu_{h,\mathsf{L}}) \leftarrow (\mathsf{msg}_{1\to h}^1, \ldots, \mathsf{msg}_{n\to h}^1) \in \{0,1\}^{\mathsf{L}}$. Denote $\mathbf{K}_{h\to i,\alpha} = \mathbf{K}_{h\to i,\alpha}^{\mu_{h,\alpha}}$, and $\mathcal{A}_j$ samples $\mathbf{K}_{h\to g}$ for $g \in \mathbb{H}$ conditioning on $\mathbf{K}_{h,\alpha} = \bigoplus_{k\in[n]} \mathbf{K}_{h\to k,\alpha}$, for every $i \in \mathcal{I}$, every $\alpha \in [\mathsf{L}]$.
- $\mathcal{A}_j$ send message $(\overline{\mathsf{GC}}_h, \overline{\mathsf{msg}}_h^2, \{\mathbf{K}_{k\to h,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$ as the second round $P2P$ message for P_h in Π_{Comp}, where $\overline{\mathsf{GC}}_h$ is the set of the garbled circuits obtained in the first round.
- Upon receiving $(\overline{\mathsf{GC}}_i, \overline{\mathsf{msg}}_{i\to j}^2, \{\mathbf{K}_{k\to i,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$ from $\mathcal{A}$, $\mathcal{A}_j$ does the following. Based on the first-round messages from P_i, denote $\mathbf{K}_{i\to h,\alpha} = \mathbf{K}_{i\to h,\alpha}^{\mu_{h,\alpha}}$, reconstruct $\mathbf{K}_{i,\alpha} \leftarrow \bigoplus_{k\in[n]} \mathbf{K}_{i\to k,\alpha}$, and compute $\mathsf{k}_i \leftarrow \mathtt{eval}(\mathtt{GC}_i, \{\mathbf{K}_{i,\alpha}\}_{\alpha\in[\mathsf{L}]})$. Then $\mathcal{A}_j$ computes $\mathsf{msg}_{i\to j}^2 \leftarrow \mathsf{k}_i \oplus \overline{\mathsf{msg}}_{i\to j}^2$ and broadcasts the message $\mathsf{msg}_{i\to j}^2$ for every corrupted party P_i in protocol Π_{bc}.
- $\mathcal{A}_j$ outputs whatever $\mathcal{A}$ outputs and halts.

By the security of Π_{bc}, for every $j \in \mathbb{H}$, there exists a PPT simulator $\mathcal{S}_j$ for the adversarial strategy $\mathcal{A}_j$, such that for every auxiliary input aux, for all $\boldsymbol{x} \in (\{0,1\}^*)^n$, for all $\lambda \in \mathbb{N}$, it holds that

$$\mathtt{REAL}_{\Pi_{\mathsf{bc}},\mathcal{I},\mathcal{A}_j(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda) \stackrel{c}{\equiv} \mathtt{IDEAL}^{\mathsf{un\text{-}abort}}_{f,\mathcal{I},\mathcal{S}_j(\mathsf{aux})}(\boldsymbol{x}, 1^\lambda)$$

Every simulator $\mathcal{S}_j$ will start by extracting corrupted parties' inputs $\boldsymbol{x}_j = \{x'_{i,j}\}_{i\in\mathcal{I}}$, and send them to the ideal functionality. Upon receiving the output value y, $\mathcal{S}_j$ can send a message $\mathtt{abort}$ or not send it to the ideal functionality.

Finally, the outputs the simulated view (in Π_{bc}) of the adversary as follows: $\mathsf{view}_j = (\hat{\mathsf{aux}}^j, \{(\hat{x}_i^j, \hat{r}_i^j)\}_{i\in\mathcal{I}}, \{\hat{\mathsf{msg}}_k^{l,j}\}_{k\in[n],l\in\{1,2\}})$.

We have the following simulator $\mathcal{S}$ (in Figure 4.3) that use $\mathcal{S}' = \mathcal{S}_j$, where j is the minimal index s.t. $j \in \mathbb{H}$, and $\mathcal{A}_j$ is the corresponding receiver specific adversary.

Figure 4.3: The Simulator $\mathcal{S}$

The simulator $\mathcal{S}$ starts by invoking $\mathcal{S}'$ on its inputs. During this interaction, $\mathcal{S}'$ could invoke the ideal functionality w.r.t., adversarial input $\boldsymbol{x}' = \{x_i'\}_{i\in\mathcal{I}}$, in this case, $\mathcal{S}$ will forward received messages to the ideal functionality receiving the output y which is forwarded to $\mathcal{S}'$. Moreover, $\mathcal{S}'$ could abort specifying a set of indices $\mathcal{H}$ and so it does $\mathcal{S}$ sending an abort message with indices $\mathcal{H}$ to the ideal functionality. In the end of the interaction $\mathcal{S}'$ produces the following simulated view: $\mathsf{view}' = (\hat{\mathsf{aux}}, \{(\hat{x}_i, \hat{r}_i)\}_{i\in\mathcal{I}}, \{\hat{\mathsf{msg}}_k^{l}\}_{k\in[n],l\in\{1,2\}})$. Then, the simulator $\mathcal{S}$ starts $\mathcal{A}$ using its inputs and correlated randomness for the corrupted parties and executes the following steps:

First round:

- $\mathcal{S}$ samples a uniformly random string $\mathbf{K}^b_{h\to i,\alpha}$ for every $i \in \mathcal{I}$, every $\alpha \in [\mathsf{L}]$ and $b \in \{0,1\}$. $\mathcal{S}$ computes $\mathsf{k}_h \xleftarrow{\$} \{0,1\}^\ell$, set f_h a constant function that take as input first-round messages $\{\mathsf{msg}_i^1\}_{i\in[n]}$, and output k_h. Then set $\mathtt{C}_h$ the boolean circuit that achieves f_h. $\mathcal{S}$ computes $\mathtt{GC}_h \leftarrow \mathtt{simGC}(1^\lambda, \phi(\mathtt{C}_h), \text{“}ckt\text{”})$. Then $\mathcal{S}$ send $\mathcal{A}$ the message $(\mathtt{GC}_h, \hat{\mathsf{msg}}_h^1, \{\mathbf{K}^b_{h\to i,\alpha}\}_{\alpha\in[\mathsf{L}]})$ on behalf of the honest party P_h to every corrupted P_i, and $\mathcal{A}$ sends back $(\mathtt{GC}_i, \mathsf{msg}_{i\to h}^1, \{\mathbf{K}^b_{i\to h,\alpha}\}_{\alpha\in[\mathsf{L}]})$.

Second round:

- If for every $i \in \mathcal{I}$ there exists a value $\hat{\mathsf{msg}}_i^1$ s.t. $\hat{\mathsf{msg}}_i^1 = \mathsf{msg}_{i\to h}^1$ for every $h \in \mathbb{H}$ (It means $\mathcal{A}$ sends the consistent messages), then
  - For every honest party P_h, compute $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \mathtt{simGC}(1^\lambda, \mathsf{k}_h, \text{“}input\text{”})$.
  - Concatenate all messages $\{\hat{\mathsf{msg}}_k^1\}_{k\in[n]}$ as: $(\mu_1, \ldots, \mu_{\mathsf{L}}) \leftarrow (\hat{\mathsf{msg}}_1^1, \ldots, \hat{\mathsf{msg}}_n^1)$. For every $i \in \mathcal{I}$ and $\alpha \in [\mathsf{L}]$, denote $\mathbf{K}_{h\to i,\alpha} \leftarrow \mathbf{K}^{\mu_\alpha}_{h\to i,\alpha}$. Then sample uniformly random strings $\mathbf{K}_{h\to o,\alpha}$ for $o \in \mathbb{H}$, conditioning on $\mathbf{K}_{h,\alpha} = \bigoplus_{k\in[n]} \mathbf{K}_{h\to k,\alpha}$.
  - Compute $\overline{\hat{\mathsf{msg}}}_h^2 \leftarrow \hat{\mathsf{msg}}_h^2 \oplus \mathsf{k}_h$. Let $\overline{\mathsf{GC}}_h$ the set of garble circuits received from the other parties in the first round. Send the message $(\overline{\mathsf{GC}}_h, \overline{\hat{\mathsf{msg}}}_h^2, \{\mathbf{K}_{k\to h,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$ to $\mathcal{A}$ as the second-round messages in Π_{Comp}.
- If there exists $i \in \mathcal{I}$ and $h, h' \in \mathbb{H}$ s.t. $\mathsf{msg}_{i\to h}^1 \neq \mathsf{msg}_{i\to h'}^1$ (It means $\mathcal{A}$ sends the inconsistent messages), then
  - Send $(\mathtt{abort}, \mathbb{H})$ to the ideal functionality.

- For every honest party P_h, compute $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \mathtt{simGC}(1^\lambda, 0^\ell, \text{“}input\text{”})$.
- For every $o \in \mathbb{H}$, let $\mathsf{msg}^1_{h\to o} \leftarrow \hat{\mathsf{msg}}^1_h$.
- Concatenate all message $\{\mathsf{msg}^1_{k\to h}\}_{k\in[n]}$ as: $(\mu_{h,1},\ldots,\mu_{h,\mathsf{L}}) \leftarrow (\mathsf{msg}^1_{1\to h},\ldots,\mathsf{msg}^1_{n\to h})$. For every $i \in \mathcal{I}$ and $\alpha \in [\mathsf{L}]$, denote $\mathbf{K}_{h\to i,\alpha} \leftarrow \mathbf{K}^{\mu_{h,\alpha}}_{h\to i,\alpha}$. Then sample uniformly random string $\mathbf{K}_{h\to o,\alpha}$ for $o \in \mathbb{H}$.
- Uniform randomly sample $\overline{\mathsf{msg}}^2_h \xleftarrow{\$} \{0,1\}^\ell$. Let $\overline{\mathsf{GC}}_h$ the set of garble circuits received from the other parties in the first round. Send the message $(\overline{\mathsf{GC}}_h, \overline{\mathsf{msg}}^2_h, \{\mathbf{K}_{k\to h,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$ to $\mathcal{A}$ as the second-round messages in Π_{Comp}.

– $\mathcal{A}$ sends back message $(\overline{\mathsf{GC}}_i, \overline{\mathsf{msg}}^2_{i\to h}, \{\mathbf{K}_{k\to i,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$. If $\mathcal{A}$ echoed inconsistent garble circuits in the second round, abort, otherwise proceeds to the next step. If $\mathcal{A}$ sends consistent first-round messages of Π_{bc}, and $\mathcal{S}'$ does not abort, based on the first-round messages from P_i, denote $\mathbf{K}_{i\to h,\alpha} = \mathbf{K}^{\mu_\alpha}_{i\to h,\alpha}$, reconstruct $\mathbf{K}_{i,\alpha} \leftarrow \bigoplus_{k\in[n]} \mathbf{K}_{i\to k,\alpha}$, compute $\mathsf{k}_i \leftarrow \mathtt{eval}(\mathtt{GC}_i, \{\mathbf{K}_{i,\alpha}\}_{\alpha\in[\mathsf{L}]})$, and compute $\mathsf{msg}^2_{i\to h} \leftarrow \overline{\mathsf{msg}}^2_{i\to h} \oplus \mathsf{k}_i$. Also, check that $\mathbf{K}_{h\to i,\alpha} = \mathbf{K}^{\mu_\alpha}_{h\to i,\alpha}$. If evaluation or check fails, this honest party P_h is added to the set $\mathcal{H}$.
– $\mathcal{S}$ sends $(\mathtt{abort}, \mathcal{H})$ to the ideal functionality. $\mathcal{S}$ will output whatever $\mathcal{A}$ outputs, and halt.

We use the following hybrid experiments to prove real-world/ideal-world indistinguishability:

– $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: In this experiment, the simulator $\mathcal{S}_0$ have the access to the internal state of the ideal functionality. $\mathcal{S}_0$ can see the honest parties input and choose their outputs. Therefore, $\mathcal{S}_0$ emulates the honest parties in the protocol Π_{Comp} based on the inputs, and also sets the output of each honest parties. $\mathtt{REAL}_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ and $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ are identically distributed.
– $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but the simulator $\mathcal{S}_1$ invokes $\mathcal{S}'$ on its inputs. During this interaction, $\mathcal{S}'$ could invoke the ideal functionality w.r.t., adversarial input $\boldsymbol{x}' = \{x'_i\}_{i\in\mathcal{I}}$, in this case, $\mathcal{S}_1$ will forward received messages to the ideal functionality receiving the output y which is forwarded to $\mathcal{S}'$. Moreover, $\mathcal{S}'$ could abort specifying a set of indices $\mathcal{H}$ and so it does $\mathcal{S}_1$ sending an abort message with indices $\mathcal{H}$ to the ideal functionality.
Then, the simulator $\mathcal{S}_1$ starts $\mathcal{A}$ using its inputs and simulated correlated randomness for the corrupted parties and emulates the honest parties in the protocol Π_{Comp}, based on their inputs, i.e., the simulator behaves as in the previous experiment. In particular, $\mathcal{S}_1$ performs the following checks. $\mathcal{S}_1$ checks if the adversary sends inconsistent garble circuits in the first round. In this case she sends $(\mathtt{abort}, \mathbb{H})$ to the ideal functionality. Moreover, $\mathcal{S}_1$ checks whether every honest party can evaluate the garbled circuits with the garbled labels from $\mathcal{A}$ in the second round, and that $\mathcal{A}$ sent the correct shares of the

honest parties' garbled labels corresponding to the first-round messages. Let $\mathcal{H}$ denote the set of honest parties for which the above checks failed, then $\mathcal{S}_1$ sends $(\texttt{abort}, \mathcal{H})$ to the ideal functionality. The indistinguishability between the hybrids follows from Lemma 3.

– $\texttt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but other than computing $\mathbf{K}^b_{h\to k,\alpha}$ for $b \in \{0,1\}, h \in \mathbb{H}, k \in [n], \alpha \in [\mathsf{L}]$, s.t. $\mathbf{K}^b_{h,\alpha} = \bigoplus_{k\in[n]} \mathbf{K}^b_{h\to k,\alpha}$, the simulator $\mathcal{S}_2$ sends random shares in the first round on behalf of every honest party P_h to every corrupted party P_i. Then:
  - If $\mathcal{A}$ sends consistent messages in the first round, it means every honest party will receive the same messages from each corrupted party, and we denoted the concatenated messages as $\{\mu_\alpha\}_{\alpha\in[\mathsf{L}]}$. Then denote $\mathbf{K}_{h\to i,\alpha} \leftarrow \mathbf{K}^{\mu_\alpha}_{h\to i,\alpha}$, and sample uniformly random string $\mathbf{K}_{h\to o,\alpha}$ for $o \in \mathbb{H}$ conditioned on $\mathbf{K}_{h,\alpha} = \bigoplus_{k\in[n]} \mathbf{K}_{h\to k,\alpha}$. These shares are used in computing the second-round messages.
  - If $\mathcal{A}$ send inconsistent messages (i.e., at least one corrupted party sends different messages to two honest parties), then denote $\{\mu_{h,\alpha}\}_{\alpha\in[\mathsf{L}]}$ as the concatenated messages, and denote $\mathbf{K}_{h\to i,\alpha} \leftarrow \mathbf{K}^{\mu_{h,\alpha}}_{h\to i,\alpha}$. Sample uniformly random string $\mathbf{K}_{h\to o,\alpha}$ for $o \in \mathbb{H}$ without conditions. These shares are used in computing the second-round messages.

  The indistinguishability between the hybrids follows from Lemma 4.

– $\texttt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but other than generating garble circuit honestly, the simulator $\mathcal{S}_3$ will use $\texttt{simGC}$ to generate garbled circuit. Specifically, in the first round, $\mathcal{S}_3$ computes $\mathsf{k}_h \xleftarrow{\$} \{0,1\}^\ell$, set f_h a constant function that take as input first-round messages $\{\mathsf{msg}^1_i\}_{i\in[n]}$, and output k_h. Then set $\mathtt{C}_h$ the boolean circuit that achieves f_h. $\mathcal{S}_3$ computes $\mathtt{GC}_h \leftarrow \texttt{simGC}(1^\lambda, \phi(\mathtt{C}_h), \text{“}ckt\text{”})$. Then, every honest party can obtain msg^1_h by using the inputs and the correlated randomness. Then compute $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \texttt{simGC}(1^\lambda, \mathsf{k}_h, \text{“}input\text{”})$. The indistinguishability between the hybrids follows from Lemma 5.

– $\texttt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but the simulator $\mathcal{S}_4$ will use different inputs for $\texttt{simGC}$ to generate garbled circuit. If at least one corrupted party sends different messages to two honest parties $\mathcal{S}_4$ computes $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \texttt{simGC}(1^\lambda, 0^\ell, \text{“}input\text{”})$ otherwise she computes $\{\mathbf{K}_{h,\alpha}\}_{\alpha\in[\mathsf{L}]} \leftarrow \texttt{simGC}(1^\lambda, \mathsf{k}_h, \text{“}input\text{”})$. The indistinguishability between the hybrids follows from Lemma 6.

– $\texttt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but when at least one corrupted party sends different messages to two honest parties, the simulator $\mathcal{S}_5$ will sample uniformly random $\overline{\mathsf{msg}}^2_h$, other than compute $\mathsf{msg}^2_h \oplus \mathsf{k}_h$. The indistinguishability between the hybrids follows from Lemma 7.

– $\texttt{Expt}^6_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$: this experiment is identical to the previous experiment but the simulator $\mathcal{S}_6$ will use the simulated messages from $\mathcal{S}'$. In the first round, P_h will send $\hat{\mathsf{msg}}^1_h$. In the second round, $\mathcal{S}_6$ computes $\overline{\mathsf{msg}}^2_h \leftarrow \hat{\mathsf{msg}}^2_h \oplus \mathsf{k}_h$. In this case, $\mathcal{S}_6$ does not need to access to the internal state of the ideal functionality, and it is exactly the same as $\mathcal{S}$, so $\texttt{Expt}^6_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ is indistin-

guishable from $\mathtt{IDEAL}^{\mathsf{sa\text{-}abort}}_{f,\mathcal{I},\mathcal{S}}$. The indistinguishability between the hybrids follows from Lemma 8.

Lemma 3. $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the security of* $\Pi_{\mathtt{bc}}$ *and the correctness of the garbling scheme.*

Proof. This lemma holds for the following reasons:

- If $\mathcal{A}$ sends inconsistent first-round messages or inconsistent garbled circuits, in $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, all honest parties will abort; in $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, $\mathcal{S}_1$ will send $(\mathtt{abort}, \mathbb{H})$ to the ideal functionality, and achieve the same results as in $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$.
- If $\mathcal{A}$ sends the consistent first-round messages, $\mathcal{A}$ can reconstruct the random value k_h for the honest party P_h and compute the second-round messages in $\Pi_{\mathtt{bc}}$, and then send garbled circuits and garbled labels to honest party. If honest parties fail to evaluate the garbled circuits then $\mathcal{A}$ sent incorrect shares for honest parties' garbled labels, they will abort, and $\mathcal{S}_1$ sends $(\mathtt{abort}, \mathcal{H})$ to the ideal functionality, where $\mathcal{H}$ contains the set of indices for which the checks fail. Otherwise, by the correctness of the garbling scheme, the honest parties will receive the correct random value k_i.
  It is important to note that $\mathcal{A}$ can send inconsistent second-round messages, for instance $\overline{\mathsf{msg}}^2$ and $\overline{\mathsf{msg}}^{2'}$, which output two different second-round messages of $\Pi_{\mathtt{bc}}$. If this is the case we could construct a reduction that violates the security of $\Pi_{\mathtt{bc}}$, which aborts in the end of the second round. Note that this reduction crucially relies on the assumption that $\mathcal{S}_1$ extracts the input of the adversary from the first round, and from the fact that $\mathcal{S}_1$ does not need the second-round messages of the adversary to generate the second-round messages of the honest party since the adversary could be rushing. Therefore, if is possible to distinguish between $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ and $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, it is possible to distinguish between the output of $\mathcal{S}'$ and one of the receiver-specific adversary $\mathcal{A}_j$, which contradicts the security of $\Pi_{\mathtt{bc}}$.

Lemma 4. $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{p}{\equiv} \mathtt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the perfect security of additive secret sharing.*

Proof. If $\mathcal{A}$ sends consistent first-round messages, then $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ is the same as $\mathtt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$. If $\mathcal{A}$ sends inconsistent first-round messages, In $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, the adversary can choose which shares (i.e., $\mathbf{K}^0_{h,\alpha}$ and $\mathbf{K}^1_{h,\alpha}$ for honest party P_h) to see, however, $\mathcal{A}$ can not recover the garbled label correctly because it does not receive a sufficient amount of shares; If $\mathcal{A}$ send inconsistent first-round messages in $\mathtt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, the adversary receives random shares that are irrelevant to the actual garbled labels. Due to the perfect security of additive secret sharing the hybrids are perfectly indistinguishable.

Lemma 5. $\mathtt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \mathtt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the privacy of garbling schemes.*

Proof. We need to prove it through $n+1$ different hybrids. In hybrid $\mathcal{H}_0$, it is equivalent to $\texttt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$. In the hybrid $\mathcal{H}_k$ for $0 < k < n$, the party P_h for $h < k$ use $\texttt{simGC}$ to obtain the garbled circuits in the first, other parties generate the garbled circuits honestly. In hybrid $\mathcal{H}_n$, it is equivalent to $\texttt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$. Assuming there exists an adversary $\mathcal{A}_{\mathcal{D}}$ can distinguish between hybrid $\mathcal{H}_k$ and hybrid $\mathcal{H}_{k+1}$, then we can construct $\mathcal{A}'_{\mathcal{D}}$ that can break the privacy of the garbling scheme by the following reduction:

- $\mathcal{A}'_{\mathcal{D}}$ sends to the challenger $\mathcal{C}$ the circuit $\mathtt{C}_k$ obtaining $\mathtt{GC}_k$.
- On behalf of other honest parties, $\mathcal{A}'_{\mathcal{D}}$ runs the first round according to both $\mathcal{H}_k$ and $\mathcal{H}_{k+1}$ while for the k-th party, $\mathcal{A}'_{\mathcal{D}}$ uses $\mathtt{GC}_k$. $\mathcal{A}'_{\mathcal{D}}$ receives first-round messages from $\mathcal{A}_{\mathcal{D}}$.
- $\mathcal{A}'_{\mathcal{D}}$ computes k_h as $\mathcal{S}_2$ would do and sends it to the $\mathcal{C}$ obtaining $\{\mathbf{K}_{k,\alpha}\}_{\alpha\in[\mathsf{L}]}$.
- On behalf of other honest parties, $\mathcal{A}'_{\mathcal{D}}$ runs the second round according to both $\mathcal{H}_k$ and $\mathcal{H}_{k+1}$ while for the k-th party, $\mathcal{A}'_{\mathcal{D}}$ uses $\{\mathbf{K}_{k,\alpha}\}_{\alpha\in[\mathsf{L}]}$.
- $\mathcal{A}'_{\mathcal{D}}$ outputs what $\mathcal{A}_{\mathcal{D}}$ outputs.

We now observe that if $\mathcal{C}$ provides the garbled circuit and the garbled labels from real execution, then we are in hybrid $\mathcal{H}_k$, otherwise, we are in hybrid $\mathcal{H}_{k+1}$. Because it works for all hybrids, it implies $\texttt{Expt}^2_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \texttt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$.

Lemma 6. $\texttt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \texttt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the partial evaluation resiliency of garbling schemes.*

Proof. This reduction is similar to the one above, where we need also $n+1$ hybrids, but every hybrids only change the garbled circuits' output. Assuming there exists an adversary $\mathcal{A}_{\mathcal{D}}$ can distinguish between hybrids $\mathcal{H}_k$ and hybrid $\mathcal{H}_{k+1}$, then $\mathcal{A}'_{\mathcal{D}}$ can break the partial evaluation resiliency of the garbling scheme by the following reduction:

- $\mathcal{A}'_{\mathcal{D}}$ sends to the challenger $\mathcal{C}$ the circuit $\mathtt{C}_k$ obtaining $\mathtt{GC}_k$.
- On behalf of other honest parties, $\mathcal{A}'_{\mathcal{D}}$ runs the first round according to both $\mathcal{H}_k$ and $\mathcal{H}_{k+1}$ while for the k-th party, $\mathcal{A}'_{\mathcal{D}}$ uses $\mathtt{GC}_k$. $\mathcal{A}'_{\mathcal{D}}$ receives first-round messages from $\mathcal{A}_{\mathcal{D}}$.
- $\mathcal{A}'_{\mathcal{D}}$ computes k_h as $\mathcal{S}_3$ would do and sends $(\mathsf{k}_h, 0^\ell)$ to $\mathcal{C}$ obtaining $\{\mathbf{K}_{k,\alpha}\}_{\alpha\in[\mathsf{L}]}$.
- On behalf of other honest parties, $\mathcal{A}'_{\mathcal{D}}$ runs the second round according to both $\mathcal{H}_k$ and $\mathcal{H}_{k+1}$ while for the k-th party, $\mathcal{A}'_{\mathcal{D}}$ uses $\{\mathbf{K}_{k,\alpha}\}_{\alpha\in[\mathsf{L}]}$.
- $\mathcal{A}'_{\mathcal{D}}$ outputs what $\mathcal{A}_{\mathcal{D}}$ outputs.

We now observe that if $\mathcal{C}$ provide the garbled circuit and the garbled labels from $\texttt{simGC}$ with circuit $\mathtt{C}_h$ and output k_h, then we are in hybrid $\mathcal{H}_k$, otherwise, we are in hybrid $\mathcal{H}_k + 1$. Because it works for all hybrids, it implies $\texttt{Expt}^3_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \texttt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$.

Lemma 7. $\texttt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{p}{\equiv} \texttt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the perfect security of one-time pad encryption.*

Proof. If $\mathcal{A}$ sends consistent first-round messages, then $\mathtt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ is the same as $\mathtt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$. If $\mathcal{A}$ sends inconsistent first-round messages, In $\mathtt{Expt}^4_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, the honest party's second-round messages $\overline{\mathsf{msg}}^2_h$ is computed by $\mathsf{msg}^2_h \oplus \mathsf{k}_h$. If $\mathcal{A}$ send inconsistent first-round messages, in $\mathtt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, honest party's second-round messages $\overline{\mathsf{msg}}^2_h$ is uniform randomly sampled. Due to the perfect security of one-time pad encryption, the hybrids are perfectly indistinguishable.

Lemma 8. $\mathtt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \mathtt{Expt}^6_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$*: This relies on the security of* $\Pi_{\mathtt{bc}}$*.*

Proof. Note that only when the adversary sends consistent messages we are generating the corresponding next honest message invoking the underlying simulator $\mathcal{S}_j$ of $\Pi_{\mathtt{bc}}$. Moreover, if $\mathcal{A}$ sends inconsistent messages in the first round $\mathcal{A}_j$ aborts. In this way, we ensure that the messages used by $\mathcal{S}'$ are the one from which $\mathcal{S}_j$ extracts the corrupted parties' inputs in the first round. Therefore, if there exists an adversary that can distinguish between $\mathtt{Expt}^5_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ and $\mathtt{Expt}^6_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, it can be used to distinguish the output between $\mathcal{S}'$ and $\mathcal{A}_j$, which violates the security of $\Pi_{\mathtt{bc}}$.

By the above demonstrations, we know $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \mathtt{Expt}^6_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, therefore we can conclude that $\mathtt{REAL}_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}} \stackrel{c}{\equiv} \mathtt{IDEAL}^{\mathsf{sa\text{-}abort}}_{f,\mathcal{I},\mathcal{S}}$.

Then work of [GIS18] provides a two-round MPC protocol in the OT correlated randomness model with malicious un-abort security and black-box access to pseudorandom generator (PRG), and black-box straight-line simulation. Then we have the following corollary:

Corollary 1. *Let* $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ *be an efficiently computable* n*-party function. Assume the existence of secure garbling schemes (Definition 1). Then by taking the malicious two-round MPC protocol from [GIS18, Theorem 7.2] as the primitive, the compiler in Figure 4.1 yield a two-round MPC protocol in the OT correlated randomness model, that securely computes* f *with malicious* sa-abort *security against* $t < n$ *corruptions, and only requires black-box use to the garbling scheme and PRG, and only uses P2P channels.*

4.2 P2P-BC, UA, Correlated Randomness Model, $t < n$

The two-round compiler described in Figure 4.1 achieves malicious unanimous abort security (against the same corruption threshold) when we assume broadcast in the second round.

Intuitively, if the second round is over broadcast, then this can be used to reach a unanimous decision (to abort or to compute the output) among the honest parties. More in detail, in this case, the additive shares are sent using a broadcast channel. This ensures to reach agreement between the honest parties and avoids that only a subset of them can evaluate the garbled circuits correctly and recover the output.

Theorem 5 (P2P-BC, UA, t < n). *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Let Π_{bc} be a BC-BC protocol that securely computes f with malicious* un-abort *security against $t < n$ corruptions. Then, assuming the existence of secure garbling schemes (Definition 1), the protocol Π_{Comp} from Figure 4.1 can compute f with malicious* un-abort *security that uses only BC channel in the second round and the first round is over P2P channels against $t < n$ corruptions, and the compiler of Figure 4.1 only makes a black-box use of the underlying MPC protocol Π_{bc} and the garbling scheme.*

Proof. The proof follows an approach similar to the one described for Theorem 4. The receiver-specific adversary behaves in the same way. The simulator is also almost identical to the one described for Theorem 4 with the exception that when an abort occurs the simulator instructs the ideal functionality to abort for all the honest parties.

The hybrids are also the same as what we have in Theorem 4. However, in the indistinguishablity of $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$ and $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, we need to argue that the honest parties either unanimously abort or recover the output correctly. It is shown by the following:

- If $\mathcal{A}$ sends inconsistent first-round messages, in $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, all honest parties abort; in $\mathtt{Expt}^1_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$, the simulator $\mathcal{S}_1$ sends `abort` to the ideal functionality, and achieve the same results as in $\mathtt{Expt}^0_{\Pi_{\mathsf{Comp}},\mathcal{I},\mathcal{A}}$.
- If $\mathcal{A}$ sends the consistent first-round messages, $\mathcal{A}$ can reconstruct the random value k_h for the honest party P_h and obtain the second-round messages in Π_{bc}, and then send garbled circuits and garbled labels to honest party. If honest parties failed to evaluate the garbled circuits, the echoed garbled circuits were inconsistent, or $\mathcal{A}$ sent incorrect shares for honest parties' garbled labels, they aborts signaling to abort to every body else. Then all the parties unanimously abort because the last round is over BC channel and every honest party receive the same messages.
- Therefore, when $\mathcal{A}$ sent inconsistent first-round messages or $\mathcal{A}$ send first-round messages but misbehave in the second-round messages, $\mathcal{S}_1$ sends `abort` to the ideal functionality. Otherwise, by the correctness of the garbling scheme, the honest parties will receive the correct random value k_i.
- Since the last round is over BC channel, $\mathcal{A}$ can no longer send inconsistent second-round messages. Therefore, in this case we do not need the simulator to extract the inputs before the second round.

Then the indistinguishability among other hybrids are exactly the same as what we have in Theorem 4.

Similar to Corollary 1, we have the following corollary:

Corollary 2. *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Assume the existence of secure garbling schemes (Definition 1). Then by taking the malicious two-round MPC protocol from [GIS18, Theorem 7.2] as the primitive, the compiler in Figure 4.1 yield a two-round MPC protocol*

in the OT correlated randomness model, that securely computes f with malicious un-abort *security against $t < n$ corruptions, and only requires black-box use to the garbling scheme and PRG, and use P2P channels in the first round and use BC channel in the second round.*

5 Black-Box Broadcast-Optimal Three-Round Compiler

5.1 P2P-P2P-P2P, SA, CRS Model, $t < n$

In this section, we describe a compiler that makes only black-box use of cryptographic primitives and turns any three-round maliciously secure MPC protocol Π_{bc} with unanimous abort over the broadcast channels into one that only relies on $P2P$ channels and achieves selective abort. Further, if Π_{bc} is secure against $t < n$ corruptions then also the compiled protocol is secure against $t < n$ corruptions. Additionally, we require that Π_{bc} admits a simulator in which the inputs are extracted in the first two rounds.

Intuitively, in this setting, it is crucial to ensure that honest parties proceed with the computation only when consistent messages for Π_{bc} are exchanged during both the first and second rounds. This approach prevents the adversary from obtaining different versions of the first and second rounds from the honest parties. Furthermore, if the adversary sends inconsistent messages in the third round, it can be viewed as an attack on the security of Π_{bc} since the parties commit their inputs in the first two rounds. In essence, the adversary's actions would at most prevent the honest parties from successfully computing the output.

To guarantee that honest parties detect the inconsistency of the first-round messages of Π_{bc}, we adopt the same approach described in Figure 4.1. In the first round, each party P_i computes the first-round messages of Π_{bc} and additive secret share the labels of a garbled circuit, which on input the first-round messages of Π_{bc} outputs a one-time pad key k_i. In the second round, each party P_i echoes the received garbled circuits and does the one-time pad encryption of the second-round messages of Π_{bc} using key k_i. Finally, parties also send the additive shares for the other parties' garble circuit based on the first-round messages of Π_{bc}. As observed in the previous section, the second-round messages of Π_{bc} is revealed only when the garbled circuits are evaluated correctly, which is the case only when consistent first-round messages of Π_{bc} are sent. Further, echoing the garbled circuit in the second round ensures that honest parties retrieve the same one-time pad keys. At this stage, the adversary could still send inconsistent second-round messages of Π_{bc} (by sending different encryptions to different honest parties), causing the honest parties to generate multiple versions of the third round. To prevent this we apply the same logic explained above a second time, such that the third-round messages of Π_{bc} are retrieved only upon receiving consistent second-round messages of Π_{bc}.

The description of the three-round compiler can be found in Figure 5.1.

Figure 5.1: The black-box broadcast-optimal three-round compiler $\Pi_{\mathsf{ThreeComp}}$

Primitives: A n-party three-round protocol $\Pi_{\mathsf{bc}} = \{(\mathtt{frst\text{-}msg}_i, \mathtt{nxt\text{-}msg}_i^2, \mathtt{nxt\text{-}msg}_i^3, \mathtt{output}_i)\}_{i\in[n]}$ that securely computes f with malicious $\mathsf{un\text{-}abort}$ security against $t < n$ corruptions, with every round of communication over BC channel. For simplicity assume that each round message has the same length and it is ℓ bits long, so each circuit has $\mathsf{L} = n \cdot \ell$ input bits.
Private input: Every party P_i has a private input $x_i \in \{0,1\}^*$.
CRS: Set up the common reference strings crs as described in Π_{bc}

First round (P2P): Every party P_i does the following:

1. Let $\mathsf{msg}_i^1 \leftarrow \mathtt{frst\text{-}msg}_i(\mathsf{crs}, x_i)$ be P_i's first-round messages in Π_{bc}.
2. Sample $\mathsf{k}_i \xleftarrow{\$} \{0,1\}^\ell$. Set f_i a constant function that take as input first-round messages $\{\mathsf{msg}_k^1\}_{k\in[n]}$, and output k_i. Then set $\mathtt{C}_i$ the boolean circuit that achieves f_i.
3. Compute $(\mathtt{GC}_i, \mathbf{K}_i) \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C}_i)$, where $\mathbf{K}_i = \{\mathbf{K}_{i,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}}$.
4. For every $\alpha \in [\mathsf{L}]$ and $b \in \{0,1\}$, sample n uniform random strings $\{\mathbf{K}_{i\to k,\alpha}^b\}_{k\in[n]}$, such that $\mathbf{K}_{i,\alpha}^b = \bigoplus_{k\in[n]} \mathbf{K}_{i\to k,\alpha}^b$.
5. Send to every party P_j the message $(\mathtt{GC}_i, \mathsf{msg}_i^1, \{\mathbf{K}_{i\to j,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$.

Second round (P2P): Every party P_i does the following:

1. If P_i does not receive a message from some other party (or an abort message), she aborts.
2. Otherwise, let $(\mathtt{GC}_j, \mathsf{msg}_{j\to i}^1, \{\mathbf{K}_{j\to i,\alpha}^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$ be the first-round messages received from P_j.
3. Compute $\mathsf{msg}_i^2 \leftarrow \mathtt{nxt\text{-}msg}_i^2(\mathsf{crs}, x_i, \{\mathsf{msg}_{k\to i}^1\}_{k\in[n]})$, and compute $\overline{\mathsf{msg}}_i^2 \leftarrow \mathsf{k}_i \oplus \mathsf{msg}_i^2$.
4. Concatenate all received messages $\{\mathsf{msg}_{k\to i}^1\}_{k\in[n]}$ as $(\mu_{i,1}^1, \ldots, \mu_{i,\mathsf{L}}^1) \leftarrow (\mathsf{msg}_{1\to i}^1, \ldots, \mathsf{msg}_{n\to i}^1) \in \{0,1\}^{\mathsf{L}}$.
5. Let $\overline{\mathsf{GC}}_i$ be the set of garbled circuits received from the other parties in the first round.
6. Sample $\mathsf{k}_i' \xleftarrow{\$} \{0,1\}^\ell$. Set f_i' a constant function that take as input second-round messages $\{\mathsf{msg}_k^2\}_{k\in[n]}$, and output k_i'. Then set $\mathtt{C}_i'$ the boolean circuit that achieves f_i'.
7. Compute $(\mathtt{GC}_i', \mathbf{K}_i') \leftarrow \mathtt{garble}(1^\lambda, \mathtt{C}_i')$, where $\mathbf{K}_i' = \{\mathbf{K}_{i,\alpha}'^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}}$.
8. For every $\alpha \in [\mathsf{L}]$ and $b \in \{0,1\}$, sample n uniform random strings $\{\mathbf{K}_{i\to k,\alpha}'^b\}_{k\in[n]}$, such that $\mathbf{K}_{i,\alpha}'^b = \bigoplus_{k\in[n]} \mathbf{K}_{i\to k,\alpha}'^b$.
9. Send to all parties the message $(\overline{\mathsf{GC}}_i, \overline{\mathsf{msg}}_i^2, \{\mathbf{K}_{k\to i,\alpha}^{\mu_{i,\alpha}^1}\}_{k\in[n],\alpha\in[\mathsf{L}]}, \mathtt{GC}_i', \{\mathbf{K}_{i\to j,\alpha}'^b\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$.

Third round (P2P): Every party P_i does the following:

1. If P_i does not receive a message from some other party (or receives an abort message), she aborts; Otherwise, let $(\overline{\mathsf{GC}}_j, \widetilde{\mathsf{msg}}^2_{j\to i}, \{\widetilde{\mathbf{K}}_{1\to j,\alpha}\}_{\alpha\in[\mathsf{L}]}, \ldots, \{\widetilde{\mathbf{K}}_{n\to j,\alpha}\}_{\alpha\in[\mathsf{L}]}, \mathtt{GC}'_j, \{\mathbf{K}'^{,b}_{j\to i,\alpha}\}_{\alpha\in[\mathsf{L}],b\in\{0,1\}})$ be the second-round messages received from party P_j.
2. Check if the set of garbled circuits $\{\overline{\mathsf{GC}}_k\}_{k\in[n]}$ echoed in second round are consistent with the garbled circuits received in first round. If this is not the case, abort.
3. For every $j \in [n]$ and $\alpha \in [\mathsf{L}]$, reconstruct each garbled label by computing $\widetilde{\mathbf{K}}_{j,\alpha} \leftarrow \bigoplus_{k\in[n]} \widetilde{\mathbf{K}}_{j\to k,\alpha}$.
4. For every $j \in [n]$, evaluate the garbled circuits as $\mathsf{k}_j \leftarrow \mathtt{eval}(\mathtt{GC}_j, \{\widetilde{\mathbf{K}}_{j,\alpha}\}_{\alpha\in[\mathsf{L}]})$. If any evaluation fails, aborts. Otherwise, compute $\mathsf{msg}^2_{j\to i} \leftarrow \widetilde{\mathsf{msg}}^2_{j\to i} \oplus \mathsf{k}_j$.
5. Compute $\mathsf{msg}^3_i \leftarrow \mathtt{nxt\text{-}msg}^3_i(\mathsf{crs}, x_i, \{\mathsf{msg}^1_{k\to i}\}_{k\in[n]}, \{\mathsf{msg}^2_{k\to i}\}_{k\in[n]})$, and compute $\overline{\mathsf{msg}}^3_i \leftarrow \mathsf{k}'_i \oplus \mathsf{msg}^3_i$.
6. Concatenate all received messages $\{\mathsf{msg}^2_{k\to i}\}_{k\in[n]}$ as $(\mu^2_{i,1}, \ldots, \mu^2_{i,\mathsf{L}}) \leftarrow (\mathsf{msg}^2_{1\to i}, \ldots, \mathsf{msg}^2_{n\to i}) \in \{0,1\}^{\mathsf{L}}$.
7. Let $\overline{\mathsf{GC}}'_i$ be the set of garbled circuits received from the other parties in the second round.
8. Send to all parties the message $(\overline{\mathsf{GC}}'_i, \overline{\mathsf{msg}}^3_i, \{\mathbf{K}'^{,\mu^2_{i,\alpha}}_{k\to i,\alpha}\}_{k\in[n],\alpha\in[\mathsf{L}]})$.

Output Computation: Every party P_i does the following:

1. If P_i does not receive a message from some other party (or receives an abort message), she aborts; Otherwise, let $(\overline{\mathsf{GC}}'_j, \widetilde{\mathsf{msg}}^3_{j\to i}, \{\widetilde{\mathbf{K}}'_{1\to j,\alpha}\}_{\alpha\in[\mathsf{L}]}, \ldots, \{\widetilde{\mathbf{K}}'_{n\to j,\alpha}\}_{\alpha\in[\mathsf{L}]})$ be the third-round messages received from party P_j.
2. Check if the set of garbled circuits $\{\overline{\mathsf{GC}}'_k\}_{k\in[n]}$ echoed in third round are consistent with the garbled circuits received in second round. If this is not the case, abort.
3. For every $j \in [n]$ and $\alpha \in [\mathsf{L}]$, reconstruct each garbled label by computing $\widetilde{\mathbf{K}}'_{j,\alpha} \leftarrow \bigoplus_{k\in[n]} \widetilde{\mathbf{K}}'_{j\to k,\alpha}$.
4. For every $j \in [n]$, evaluate the garbled circuits as $\mathsf{k}'_j \leftarrow \mathtt{eval}(\mathtt{GC}'_j, \{\widetilde{\mathbf{K}}'_{j,\alpha}\}_{\alpha\in[\mathsf{L}]})$. If any evaluation fails, aborts. Otherwise, compute $\mathsf{msg}^3_{j\to i} \leftarrow \widetilde{\mathsf{msg}}^3_{j\to i} \oplus \mathsf{k}'_j$.
5. Compute and output $y \leftarrow \mathtt{output}_i(\mathsf{crs}, x_i, \{\mathsf{msg}^1_{k\to i}\}_{k\in[n]}, \{\mathsf{msg}^2_{k\to i}\}_{k\in[n]}, \{\mathsf{msg}^3_{k\to i}\}_{k\in[n]})$.

Then we have the following theorem.

Theorem 6 (P2P-P2P-P2P, SA, t < n). *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Let Π_{bc} be a BC-BC-BC protocol that securely computes f with malicious* un-abort *security against $t < n$ cor-*

ruptions with the additional constraint that a simulator straight-line extract the inputs before the last round. Then, assuming the existence of secure garbling schemes (Definition 1), the protocol $\Pi_{\mathsf{ThreeComp}}$ from Figure 5.1 can compute f with malicious sa-abort *security that uses only P2P channels against $t < n$ corruptions, and the protocol only makes a black-box use of the underlying MPC protocol Π_{bc} and the garbling scheme.*

Proof (sketch). The proof is very similar to the proof of Theorem 4.

Intuitively the receiver-specific adversary follows the same logic described in Figure 4.2 but extending it to the three rounds case. Roughly speaking, in this case there is a second set of garbled circuits used to recover the third-round messages. Therefore, the receiver-specific adversary applies the strategy described in Figure 4.2 twice to generate also the appropriate third-round messages in the execution of Π_{bc} (and in the internal execution of $\Pi_{\mathsf{ThreeComp}}$ with the adversary that attacks $\Pi_{\mathsf{ThreeComp}}$).

The simulator applies the same simulation strategy applied in Figure 4.3 to recover the messages of Π_{bc} extending it to three round case. Also, the remaining steps of the simulation are very similar, but in this case, there are two sets of garbled circuits, namely to recover the keys for decrypting the second round and to decrypt the last round. Therefore, $\mathcal{S}$ applies the logic described in Figure 4.3 twice to make sure to use the simulated messages of Π_{bc} only when consistent first-round messages or second-round messages of Π_{bc} are sent by the adversary. If this is not the case $\mathcal{S}$ makes sure that these messages are not recovered using the same simulation strategy of Figure 4.3.

The hybrids and indistinguishability game then follow a similar process of the proof of Theorem 4. In this case, the proof is extended with a second set of hybrids which is needed to simulate the third-round messages of Π_{bc} and switch to the second set of simulated garbled circuits (also in this case the way on which the garbled circuits and corresponding message of Π_{bc} are simulated depends on if the adversary sends consistent messages of Π_{bc}).

Then, [IKSS22b] provides a three-round MPC protocol in the CRS model with malicious un-abort security and black-box access to a malicious two-round OT, and black-box straight-line simulation. Then we have the following corollary:

Corollary 3. *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Assume the existence of secure garbling schemes (Definition 1). Then by taking the malicious three-round MPC protocol from [IKSS22b, Theorem 7.1] as the primitive, the compiler in Figure 5.1 yield a three-round MPC protocol in the CRS model, that securely computes f with malicious* sa-abort *security against $t < n$ corruptions, and only requires black-box use to the garbling scheme and a malicious two-round OT in the CRS model, and only use the P2P channels.*

5.2 P2P-P2P-BC, UA, CRS Model, $t < n$

The three-round compiler described in Figure 5.1 achieves unanimous abort security (against the same corruption threshold) when we assume the first two rounds over peer-to-peer channels, and the third round over broadcast channels.

Intuitively, in the three-round protocol, if the third round is over broadcast, then every honest party receives the same messages from the adversary in the third round. Then, if the adversary causes an abort in the third round honest parties can detect it. More in detail, if the third round is over broadcast, all honest parties receive the same encrypted third-round messages and the same set of shares of garbled labels from the corrupted parties. Then if the evaluation or check fails, all honest parties will abort. If the evaluation is successful, all the honest parties get the same third-round messages. Then we have the following theorem.

Theorem 7 (P2P-P2P-BC, UA, t < n). *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Let Π_{bc} be a BC-BC-BC protocol that securely computes f with malicious* un-abort *security against $t < n$ corruptions. Then, assuming the existence of secure garbling schemes (Definition 1), the protocol $\Pi_{\mathsf{ThreeComp}}$ from Figure 5.1 can compute f with malicious* un-abort *security that uses only BC channel in the third round and the first two round is over P2P channels against $t < n$ corruptions, and the compiler of Figure 5.1 only makes a black-box use of the underlying MPC protocol Π_{bc} and the garbling scheme.*

Proof (sketch). In a nutshell, the major difference w.r.t., the proof of Theorem 6 is that, since the third round is over the broadcast channel, the adversary can no longer send inconsistent third-round messages (more specifically, the encrypted third-round messages of the underlying MPC protocol Π_{bc}) to different honest parties. Therefore, if the honest parties do not abort unanimously in the third round, all honest party receives the same encrypted third-round messages and then they can decrypt the encrypted message to obtain the same third-round messages from Π_{bc}. Moreover, the compiler ensures that the honest parties' messages of Π_{bc} are revealed only when the adversary sends consistent first two rounds.

Therefore, we can use the same strategy as in the proof of Theorem 6 to design the receiver-specific adversary and the simulator. However, the difference between the simulator from the proof of Theorem 6 is that when the abort happens after receiving the adversary's third-round messages, the simulator instructs the ideal functionality to abort for all honest parties. Further, the hybrids and the indistinguishability game also follow a similar process of the proof of Theorem 6 where a second set of hybrids is needed to simulate the third-round messages of Π_{bc} and switch to the second set of simulated garbled circuits, and the shares of garbled labels based on whether the adversary sends consistent second-round messages or not.

Similar to Corollary 3, we have the following corollary:

Corollary 4. *Let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be an efficiently computable n-party function. Assume the existence of secure garbling schemes (Definition 1). Then by taking the malicious three-round MPC protocol from [IKSS22b, Theorem 7.1] as the primitive, the compiler in Figure 5.1 yield a three-round MPC protocol in the CRS model, that securely computes f with malicious* un-abort *security against $t < n$ corruptions, and only requires black-box use to the garbling scheme and a malicious two-round OT in the CRS model, and uses P2P channels in the first two round and uses BC channel in the third round.*

References

ABG+20. Benny Applebaum, Zvika Brakerski, Sanjam Garg, Yuval Ishai, and Akshayaram Srinivasan. Separating two-round secure computation from oblivious transfer. In Thomas Vidick, editor, *ITCS 2020: 11th Innovations in Theoretical Computer Science Conference*, volume 151, pages 71:1–71:18, Seattle, WA, USA, January 12–14, 2020. Leibniz International Proceedings in Informatics (LIPIcs).

ACGJ18. Prabhanjan Ananth, Arka Rai Choudhuri, Aarushi Goel, and Abhishek Jain. Round-optimal secure multiparty computation with honest majority. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 395–424, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

AL07. Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. In Salil P. Vadhan, editor, *TCC 2007: 4th Theory of Cryptography Conference*, volume 4392 of *Lecture Notes in Computer Science*, pages 137–156, Amsterdam, The Netherlands, February 21–24, 2007. Springer Berlin Heidelberg, Germany.

BGJ+18. Saikrishna Badrinarayanan, Vipul Goyal, Abhishek Jain, Yael Tauman Kalai, Dakshita Khurana, and Amit Sahai. Promise zero knowledge and its applications to round optimal MPC. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 459–487, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

BHR12. Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Adaptively secure garbling with applications to one-time programs and secure outsourcing. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology – ASIACRYPT 2012*, volume 7658 of *Lecture Notes in Computer Science*, pages 134–153, Beijing, China, December 2–6, 2012. Springer Berlin Heidelberg, Germany.

BL18. Fabrice Benhamouda and Huijia Lin. k-round multiparty computation from k-round oblivious transfer via garbled interactive circuits. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 500–532, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

CCG+20. Arka Rai Choudhuri, Michele Ciampi, Vipul Goyal, Abhishek Jain, and Rafail Ostrovsky. Round optimal secure multiparty computation from

minimal assumptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 291–319, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland.

CDR+23. Michele Ciampi, Ivan Damgård, Divya Ravi, Luisa Siniscalchi, Yu Xia, and Sophia Yakoubov. Broadcast-optimal four-round MPC in the plain model. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023: 21st Theory of Cryptography Conference, Part II*, volume 14370 of *Lecture Notes in Computer Science*, pages 3–32, Taipei, Taiwan, November 29 – December 2, 2023. Springer, Cham, Switzerland.

CDvdG88. David Chaum, Ivan Damgård, and Jeroen van de Graaf. Multiparty computations ensuring privacy of each party's input and correctness of the result. In Carl Pomerance, editor, *Advances in Cryptology – CRYPTO'87*, volume 293 of *Lecture Notes in Computer Science*, pages 87–119, Santa Barbara, CA, USA, August 16–20, 1988. Springer Berlin Heidelberg, Germany.

CGZ20. Ran Cohen, Juan A. Garay, and Vassilis Zikas. Broadcast-optimal two-round MPC. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 828–858, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

COSW23. Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Hendrik Waldner. List oblivious transfer and applications to round-optimal black-box multiparty coin tossing. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 459–488, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

DMR+21. Ivan Damgård, Bernardo Magri, Divya Ravi, Luisa Siniscalchi, and Sophia Yakoubov. Broadcast-optimal two round MPC with an honest majority. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 155–184, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

DRSY23. Ivan Damgård, Divya Ravi, Luisa Siniscalchi, and Sophia Yakoubov. Minimizing setup in broadcast-optimal two round MPC. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 129–158, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.

DS83. Danny Dolev and H. Raymond Strong. Authenticated algorithms for byzantine agreement. *SIAM J. Comput.*, 12(4):656–666, 1983.

FL82. Michael J. Fischer and Nancy A. Lynch. A lower bound for the time to assure interactive consistency. *Inf. Process. Lett.*, 14(4):183–186, 1982.

GIS18. Sanjam Garg, Yuval Ishai, and Akshayaram Srinivasan. Two-round MPC: Information-theoretic and black-box. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part I*, volume 11239 of *Lecture Notes in Computer Science*, pages 123–151, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland.

GL05. Shafi Goldwasser and Yehuda Lindell. Secure multi-party computation without agreement. *Journal of Cryptology*, 18(3):247–287, July 2005.

GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In

Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press.

GS18. Sanjam Garg and Akshayaram Srinivasan. Two-round multiparty secure computation from minimal assumptions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 468–499, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

HHPV18. Shai Halevi, Carmit Hazay, Antigoni Polychroniadou, and Muthuramakrishnan Venkitasubramaniam. Round-optimal secure multi-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 488–520, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

HIK+19. Shai Halevi, Yuval Ishai, Eyal Kushilevitz, Nikolaos Makriyannis, and Tal Rabin. On fully secure MPC with solitary output. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part I*, volume 11891 of *Lecture Notes in Computer Science*, pages 312–340, Nuremberg, Germany, December 1–5, 2019. Springer, Cham, Switzerland.

HL10. Carmit Hazay and Yehuda Lindell. *Efficient Secure Two-Party Protocols: Techniques and Constructions*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2010.

IKSS21. Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. On the round complexity of black-box secure MPC. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 214–243, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

IKSS22a. Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. Round-optimal black-box protocol compilers. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 210–240, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

IKSS22b. Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. Round-optimal black-box secure computation from two-round malicious OT. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part II*, volume 13748 of *Lecture Notes in Computer Science*, pages 441–469, Chicago, IL, USA, November 7–10, 2022. Springer, Cham, Switzerland.

IKSS23. Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. Round-optimal black-box MPC in the plain model. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 393–426, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

KO04. Jonathan Katz and Rafail Ostrovsky. Round-optimal secure two-party computation. In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 335–354, Santa Barbara, CA, USA, August 15–19, 2004. Springer Berlin Heidelberg, Germany.

MW16. Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key FHE. In Marc Fischlin and Jean-Sébastien Coron, edi-

tors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 735–763, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.

Ode09. Goldreich Oded. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, USA, 1st edition, 2009.

PS21. Arpita Patra and Akshayaram Srinivasan. Three-round secure multiparty computation from black-box two-round oblivious transfer. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 185–213, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

RTV04. Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Notions of reducibility between cryptographic primitives. In Moni Naor, editor, *TCC 2004: 1st Theory of Cryptography Conference*, volume 2951 of *Lecture Notes in Computer Science*, pages 1–20, Cambridge, MA, USA, February 19–21, 2004. Springer Berlin Heidelberg, Germany.

Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th Annual Symposium on Foundations of Computer Science*, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press.